

Topology Optimization

Lecture 6 – Introduction to Topology Optimization

The idea

Filtering and Projection

Implementation using python and FEniCS.

Course overview

No.	Topics
1	Introduction Function and vector spaces, Interpolation, L^2 projection
2	The Finite Element Method for PDEs 1: Poisson's Equation, The Finite Element Method, Properties of the Finite Element Method
3	The Finite Element Method for PDEs 2: Index notation, Linear Elasticity, Non-linear Poisson's equation, Non-linear Elasticity
4	Constrained Optimization 1: Optimality, KKT Theorem, Numerical Optimization Algorithms
5	Constrained Optimization 2: Method of Moving Asymptotes, MPI and PETSc
6	Introduction to Topology Optimization: The idea, Filtering and Projection, Python implementation
7	Gradients
8	Robust topology optimization, More on MPI
9	Wave equations
10	Acoustic waves and more on Robust Optimization
11	Penalty and Augmented Lagrangian Methods Stress-constraints in Topology Optimization 1
12	Stress-constraints in Topology Optimization 2
13	Light concentration using nanoparticle design
14	Heat conduction

Some mathematical prerequisites ✓ **DONE!**

Mathematical and numerical formulation of the Finite Element Method ✓ **DONE!**

Optimization Theory and numerical optimization methods ✓ **DONE!**

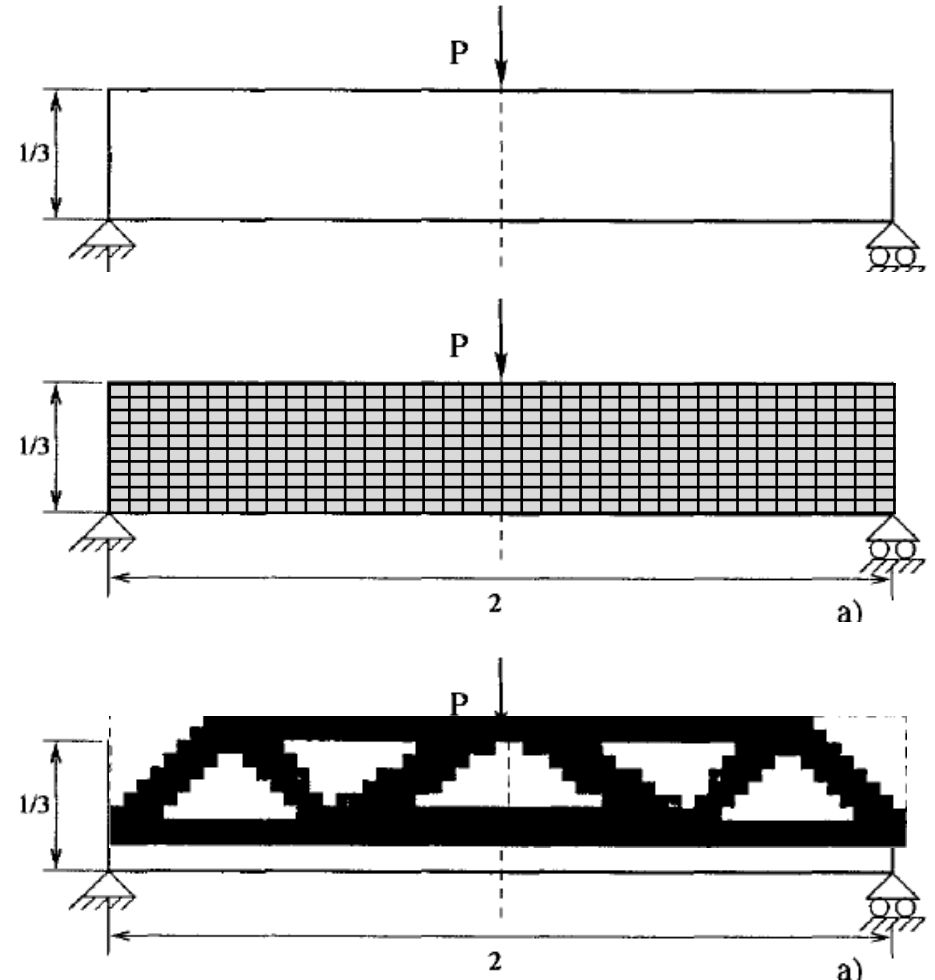
Basic Topology Optimization

Advanced Topology Optimization

Applications of Topology Optimization

The Idea

- In topology optimization, one defines a design area or volume, named the design space.
- An area is used for 2D problems.
- For 3D problems, a volume is used.
- This design space is divided into elements (or pixels or voxels).
- Each element of the design space can either contain material or be empty.
- An empty element has a “density” of 0 and an element containing material has a “density” of 1.
- An objective function is minimized with respect to the element densities, acting as “design variables”.
- The example to the right is the most used example, named the MBB beam.
- Messerschmitt-Bölkow-Blohm ([MBB](#)) was a West German aerospace manufacturer and was closed in 1989. Some of its assets are part of Air Bus.



The Idea

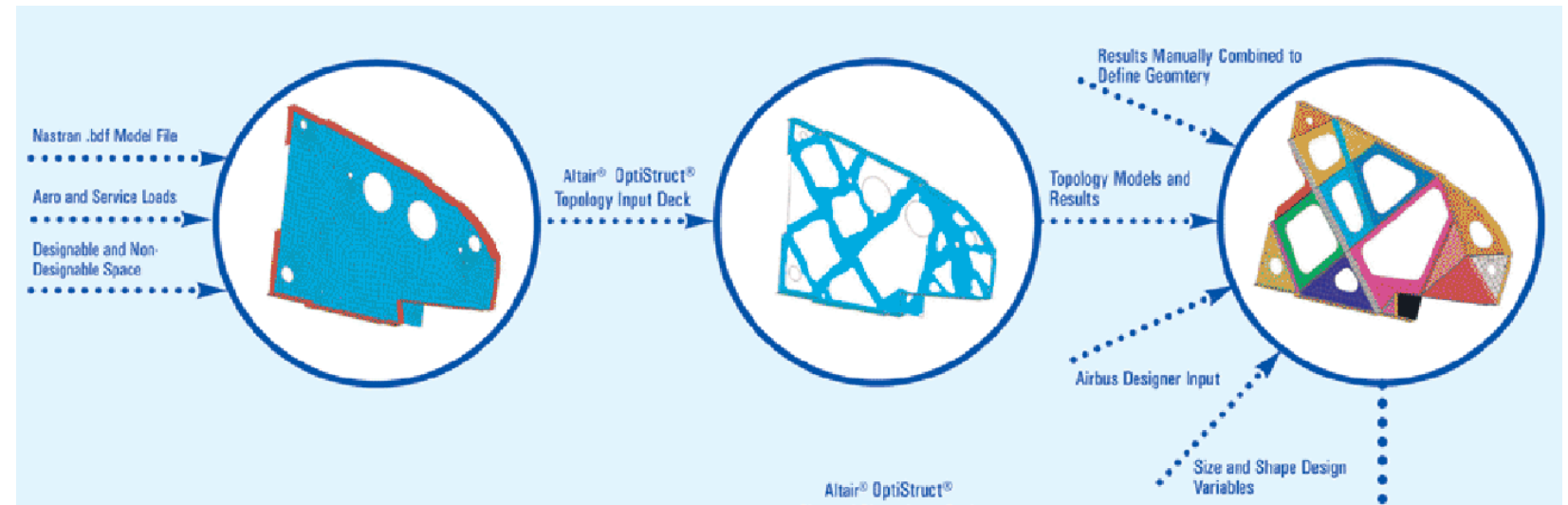
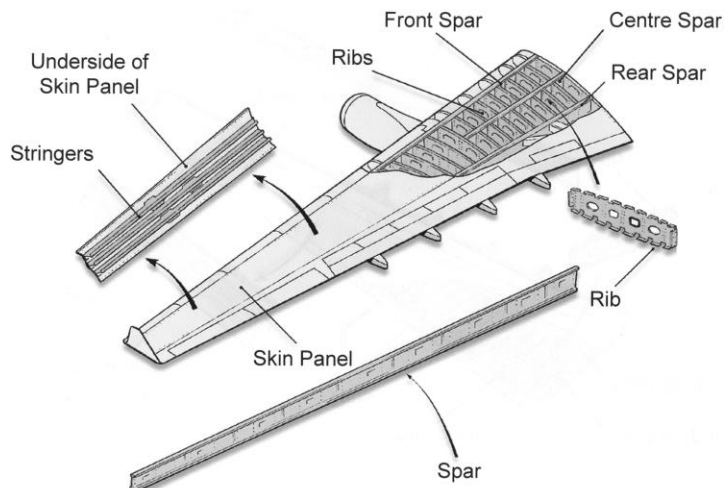
- Additional constraints are often added to the problem, for ex. an upper bound on the material use.
- Since each element contain a design variable, the problem contain hundreds, thousands, millions or billions of design variables.
- The computational geometry does not change during the computation, the mesh stays the same.
- The gradient can be computed efficiently, as we will see next week.
- Gradient based optimization algorithms are most often used to solve the constrained optimization problem.

Classic Example - Airbus A380

- Nose and wings contain internal rib structure.
- Optimizing the ribs have reduced the weight of the aircraft by about 1000 kg.
- In the design process of the A380, both parameter and topology optimization was used.
- Topology optimization was used to obtain a good “guess” of a design.
- This design was afterwards parametrized and optimized with respect to the design parameters.
- In the parametric optimization, additional things were considered which was not possible (at the time) to directly include in the topology optimization, e.g., buckling load.



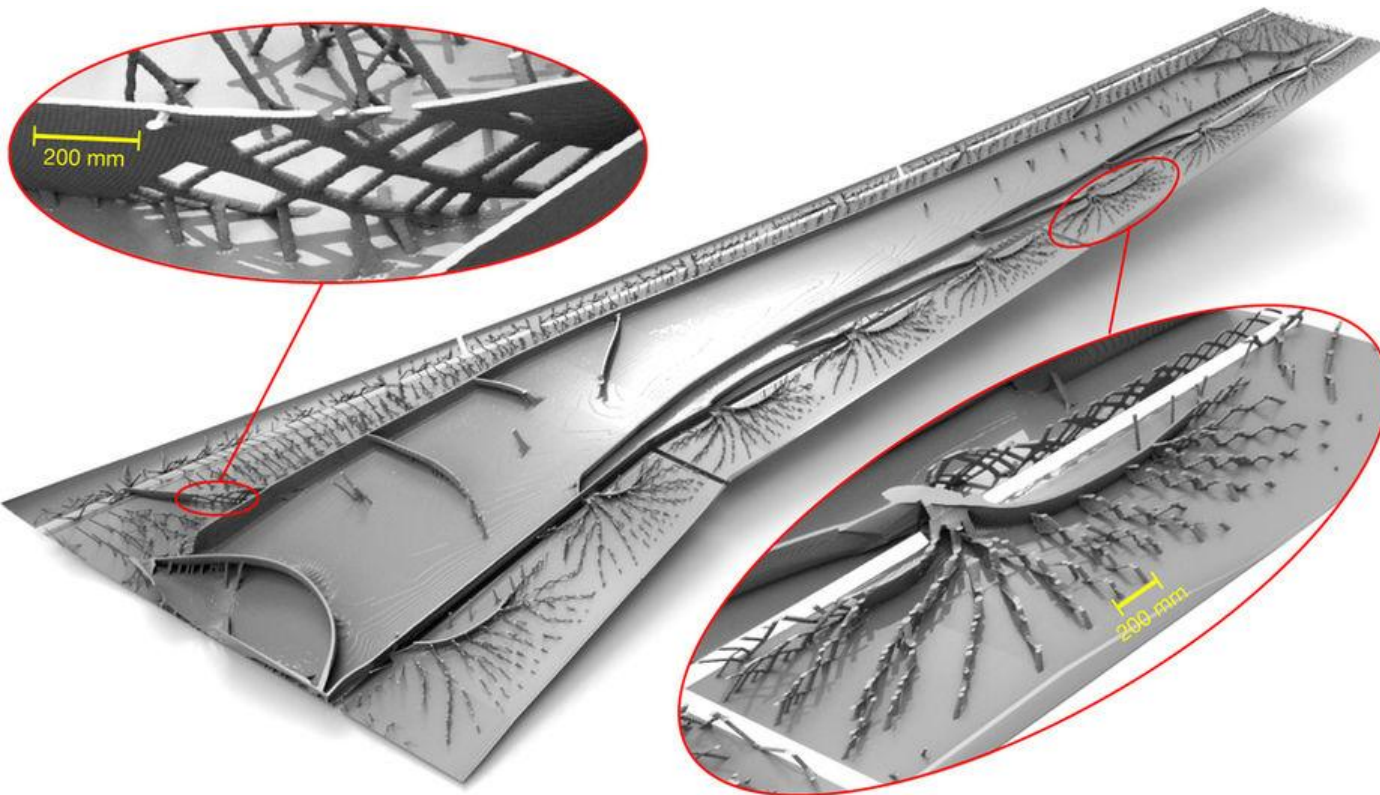
L. Krog, Altair Engineering (2002).
Application of topology, sizing, and shape optimization
methods to optimal design of aircraft components.



Large scale topology optimization from DTU

- Topology Optimization of complete full-size airplane wing.
- <https://vidensportal.deic.dk/node/176>
- 200-500 kg (2%-5%) pr. wing weight can be saved.

- <https://www.nature.com/articles/nature23911>
- Elements $1 \text{ cm}^3 \Rightarrow \sim 1$ billion variables.
- Parallel computations with ~ 10.000 cores.
- MPI/PETSc/C++ implementation.



Down-scaled and 3D printed section.

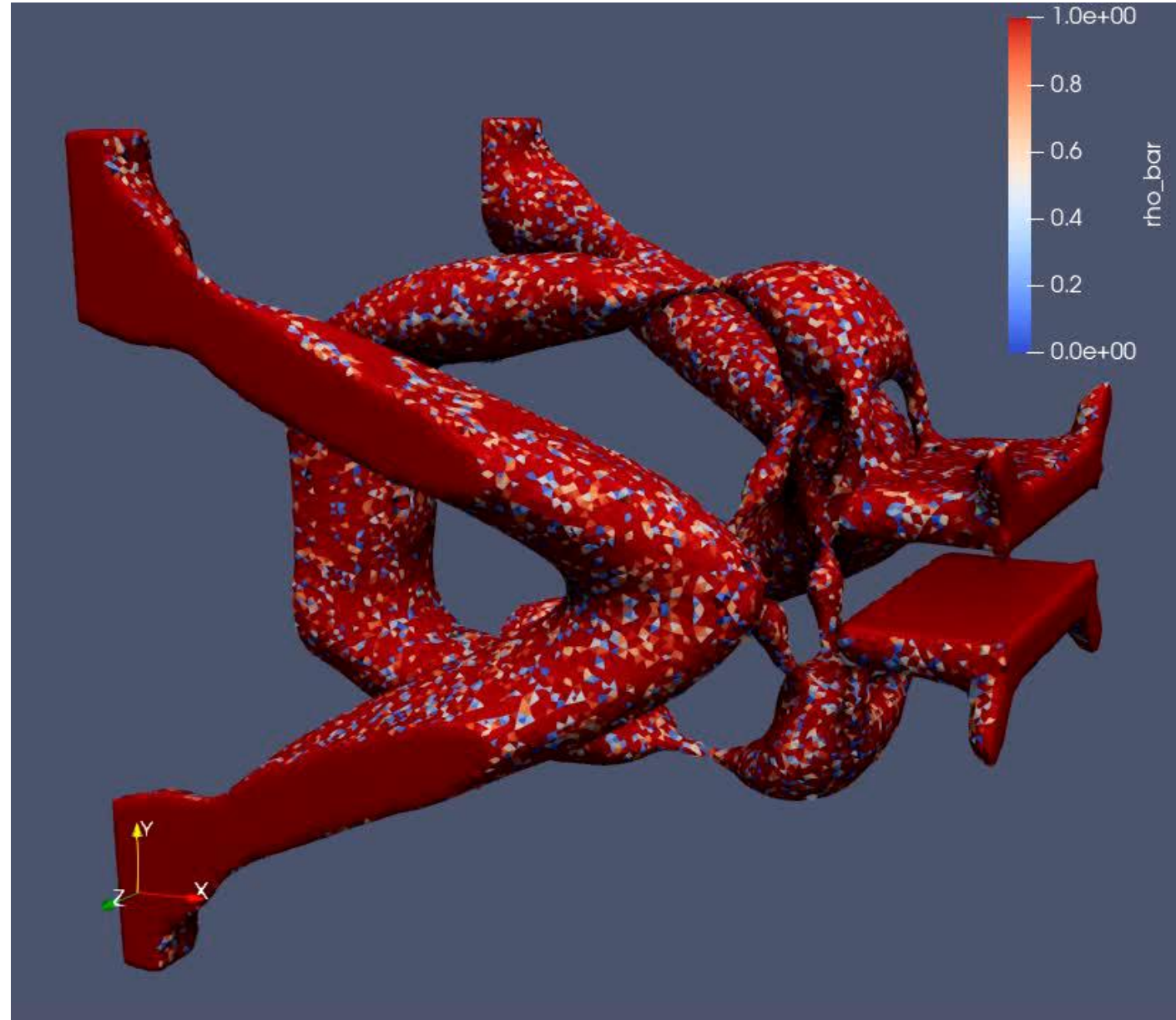


Hornbill bird beak with internal structure similar to that of the optimized aircraft wing.



Example - could be an exam project in this course 😊

- 3D compliant mechanism gripper.
- Optimized for maximum “grip” displacement.
- $\frac{1}{4}$ simulated, $\sim 750,000$ DOF.
- “Robust” optimization utilized – to be discussed later.
- Some possible additional questions:
 - What about stress?
 - What about fatigue?
 - Can it be 3D printed?
 - Will it work in real life?
 - Can the material usage be lowered?



Maximum Stiffness

- The classical problem in topology optimization is to optimize for maximum stiffness with a constraint on the used mass.
- The equilibrium equations can be discretized using the finite element method.
- The optimization problem can then be formulated as:

$$\min_{\rho_i} F(\rho(\mathbf{r})) = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} d\Omega \quad \left(\text{often used short notation for: } \min_{\rho_i} F(\rho(\mathbf{r})) \text{ where } F(\rho(\mathbf{r})) = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} d\Omega \right)$$

$$\text{such that } \sigma_{ij,j} = f_i \text{ in } \Omega, \quad \sigma_{ij} n_j = T_i \text{ on } \partial\Omega^N, \quad u_i = u_i^D \text{ on } \partial\Omega^D$$

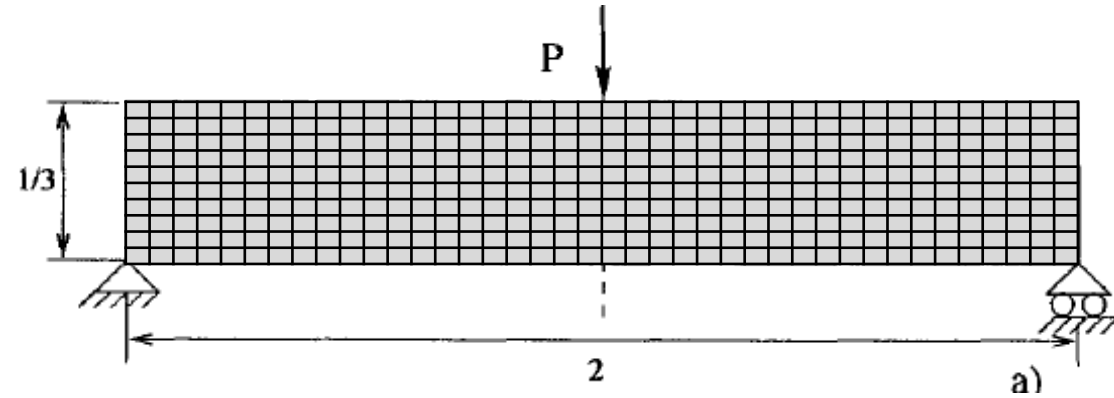
$$\epsilon_{ij} = \frac{1}{2} (u_{i,j} + u_{j,i})$$

$$\sigma_{ij} = \lambda(E) \epsilon_{kk} \delta_{ij} + 2\mu(E) \epsilon_{ij}$$

$$E(\mathbf{r}) = \rho(\mathbf{r}) E_{\text{material}}$$

$$\rho(\mathbf{r}) = 0 \text{ or } \rho(\mathbf{r}) = 1$$

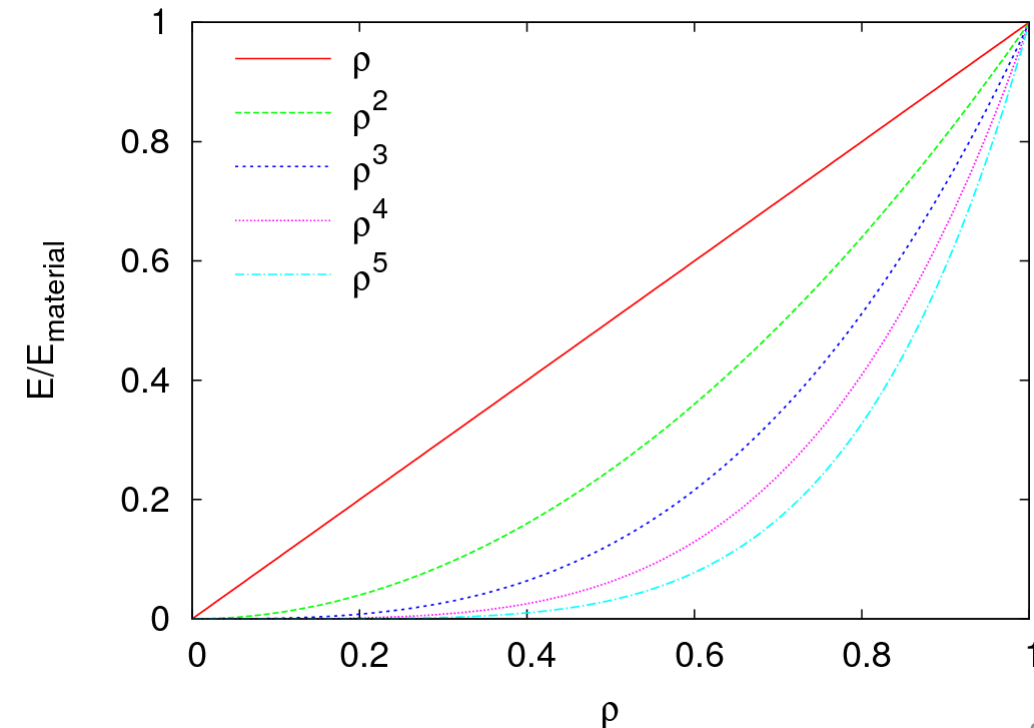
$$\int_{\Omega} \rho(\mathbf{r}) d\Omega \leq V_{\text{max}}$$



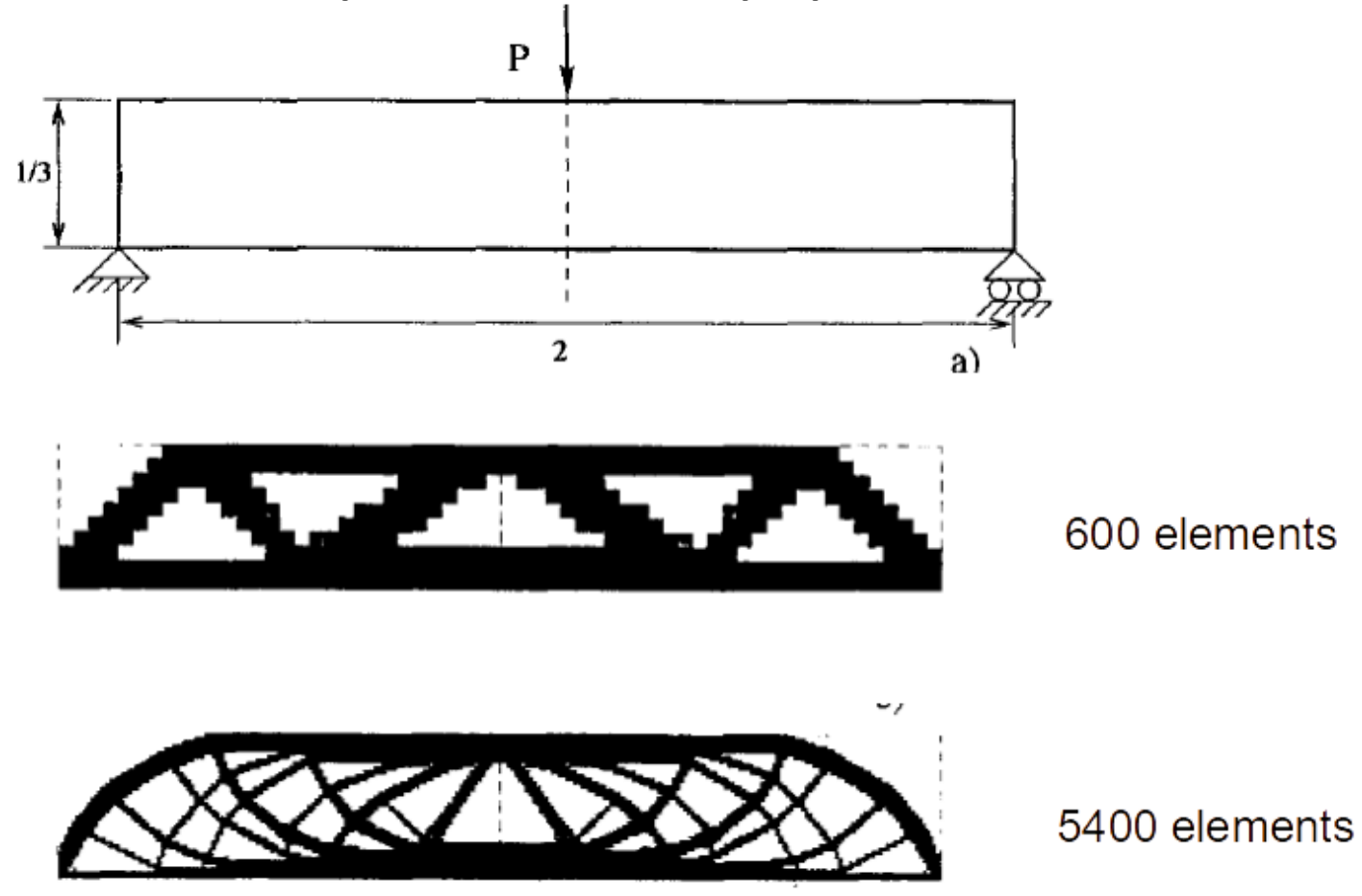
- $\rho(\mathbf{r})$ is the spatially dependent "design density" with degrees of freedom ρ_i , for example, being the expansion coefficients in the finite element discretization of $\rho(\mathbf{r})$. Often piecewise constant functions (DG0) are used.
- $E(\rho(\mathbf{r}))$ is the spatially dependent modulus of elasticity.
- $\rho(\mathbf{r}_0) = 1$ corresponds to material at spatial position $\mathbf{r}_0$.
- $\rho(\mathbf{r}_0) = 0$ corresponds to no material at spatial position $\mathbf{r}_0$.
- $F(\rho(\mathbf{r})) = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} d\Omega$ is the strain energy.
- The idea is to make the stress and/or strain as small as possible under the given loading using only a maximum amount of material.

Discrete Density Problem

- The design variables are discrete (0 or 1), so we can not differentiate F wrt. ρ_i .
- The design variables are therefore allowed to be continuous.
- A penalization scheme is added to make the optimization problem result in either 0 or 1 values of the design variables.
- This can be done using a so-called SIMP model. SIMP is short for Solid Isotropic Material with Penalization, $E(\rho) = E_{material}\rho^p$.
- A “modified SIMP” model is often more convenient:
$$E(\rho) = E_{min} + (E_{material} - E_{min})\rho^p$$
- p is a “penalty exponent”, which for $p \geq 2$ makes areas with $0 < \rho < 1$ “less stiff”.
- These areas still use the same fraction of the maximum allowed material volume and thus, become less attractive when $p > 1$.
- $p = 3$ or higher work well in practice, but too high p makes the optimization problem difficult to solve.
- In practice, $3 \leq p \leq 7$ is often used.

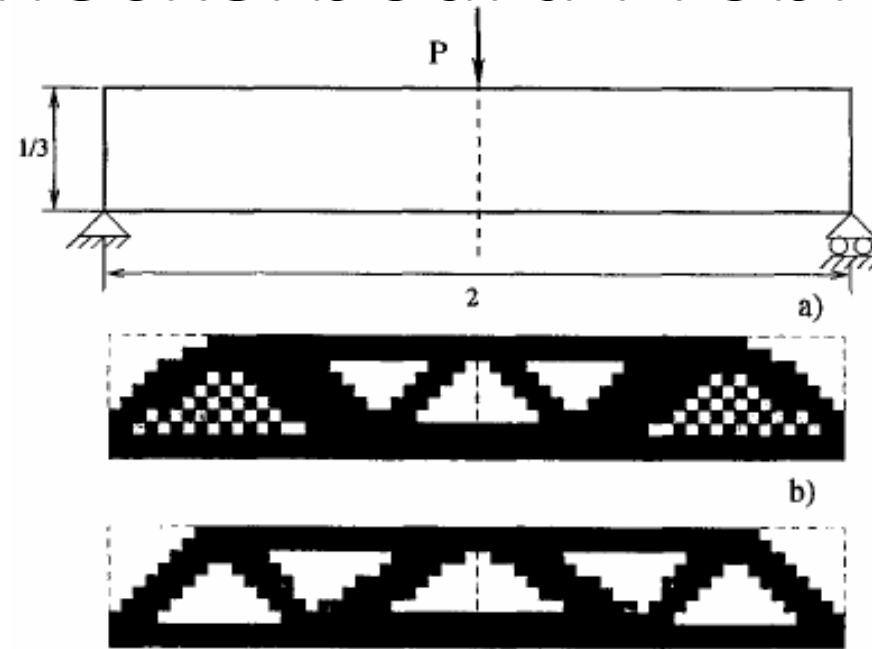


Mesh dependency problem



- Refining the mesh in the problem will result in a structure with more, but smaller, trusses.
- The solution will generate a microstructure as the mesh is refined.
- For extremely small features, the trusses made using a real material can show large displacements and strains and/or buckling and linear elasticity will not hold.

Checkerboard Problem



- Sometimes checkerboard patterns appear.
- The cause is the finite element model which overestimates the stiffness of a checkerboard pattern.
- The checkerboard is connected through the 4 corner nodes in each element, but a “real checkerboard” is not physically connected.
- **Filtering techniques cures both the checkerboard and mesh dependency problems!**

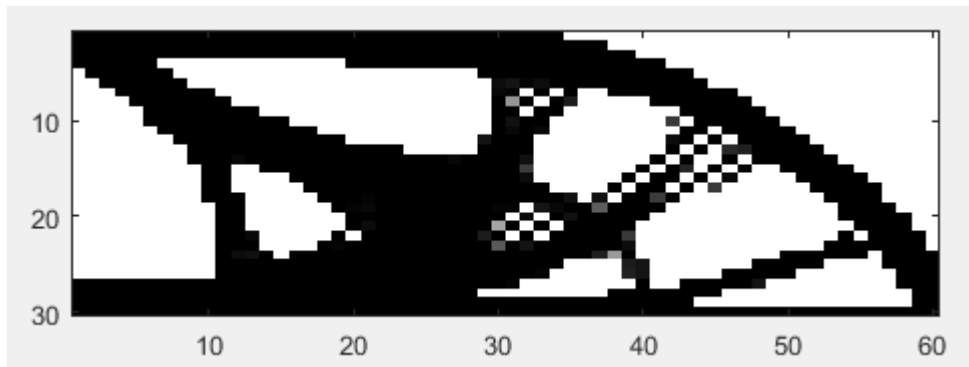
Filtering – PDE filter (Helmholtz filter)

- Both the mesh dependency problem and the checkerboard patterns are due to small features.
- Introducing a length scale into the problem such that no features can be small that this length scale will fix both problems.
- The Helmholtz equation can be used as a filter:

$$-r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} = \rho \quad , \quad \frac{\partial \tilde{\rho}}{\partial \mathbf{n}} = 0$$

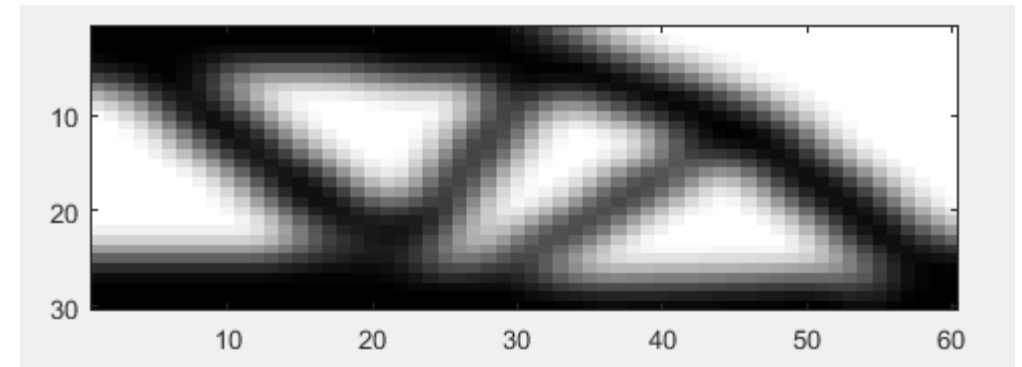
where ρ is used as the input to calculate the smoothed version, $\tilde{\rho}$.

- r is the Helmholtz filter radius and introduces a length scale in the problem.
- The filtered density is used as the "physical" density and enters e.g. in the modified SIMP model $E(\rho) = E_{\min} + (E_{\text{material}} - E_{\min}) \tilde{\rho}^p$ and in the volume constraint $\int_{\Omega} \tilde{\rho} d\Omega \leq V_{\max}$.
- The Helmholtz filter works well for both 2D and 3D problems and is easy to implement in fenics and we will use it a lot in this course.



Full optimization without filtering

Only half of
the beam is
shown.



Full optimization with Filtering

Filtering – Convolution Filter

- Instead of solving the Helmholtz PDE, one can also calculate the smoothed density for element e as

$$\tilde{\rho}_e = \frac{1}{\sum_i H_{ei}} \sum_i H_{ei} \rho_i \quad ,$$
$$H_{ei} = \max \left(0, \frac{r_{min} - \Delta(e, i)}{r_{min}} \right)$$

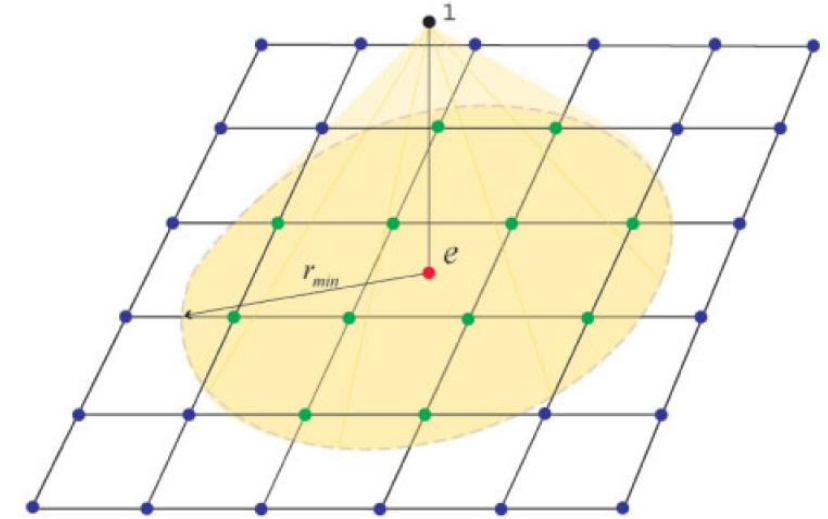
- The expression is a (discrete) convolution.
- $\Delta(e, i)$ is the center-to-center distance of elements e and i .
- r_{min} is the filter radius: If $\Delta(e, i) > r_{min}$, $H_{ei} = 0$.
- The discrete convolution can be calculated efficiently using the convolution theorem:

$$\tilde{\rho} = H * \rho = DFT^{-1} (DFT(H) \cdot DFT(\rho))$$

$*$ is the convolution.

DFT is the Discrete Fourier transform (often calculated using a FFT).

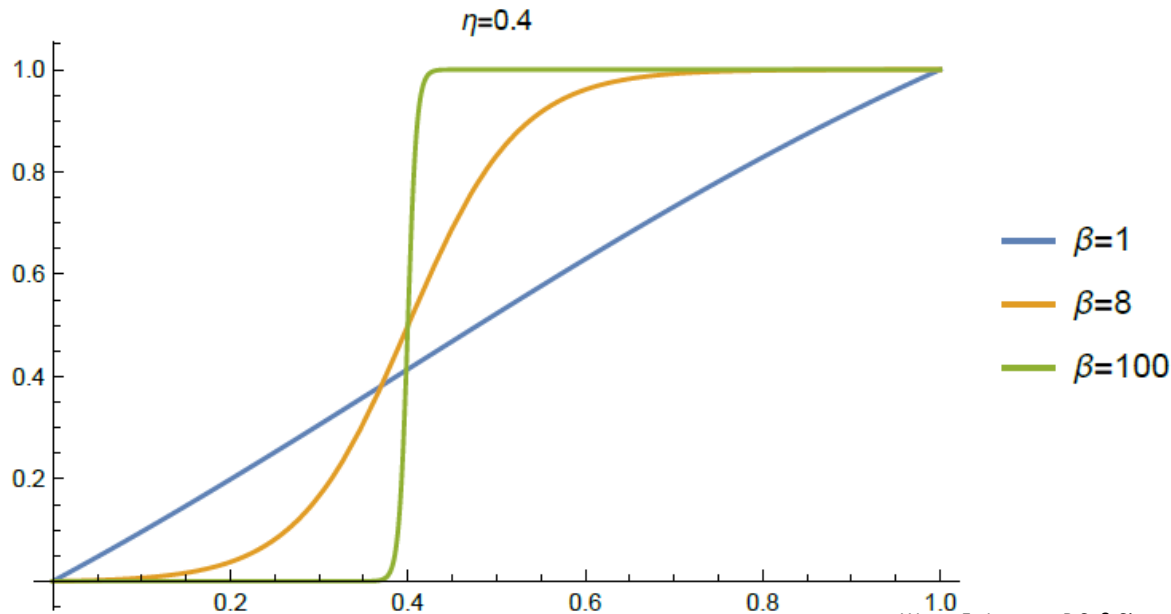
- This works very well in 2D but is less efficient in 3D than solving the Helmholtz equation.
- It can be shown the the convolution filter radius is similar to the Helmholtz filter, if one chooses the Helmholtz filter radius as $r = r_{min}/(2\sqrt{3})$.



J. K. Guest, J. H. Prévost, and T. Belytschko.
“Achieving minimum length scale in topology optimization using nodal design variables and projection functions”. In: *International Journal for Numerical Methods in Engineering* 61.2 (2004), pp. 238–254. doi: 10.1002/nme.1064.

Projection

- Since the filtering smooths the densities, we don't get nice 0-1 designs after filtering.
- The filtering should therefore be followed by a projection.
- A projection is done by a function which takes the filtered densities and "projects" them to values close to either 0 or 1.
- A natural choice for projection function is the Heaviside step function, but the function should be smooth such that we can differentiate it, so the Heaviside step function can not be used directly.
- A smoothed Heaviside function, $H(\beta, \eta, \tilde{\rho})$, is shown below, where the steepness can be controlled by β and the projection center by η .
- The projection is done in a continuation scheme, such that initially almost no projection is used but as the final geometry is approached, the projection becomes stronger and stronger by increasing β .



$$H(\beta, \eta, \tilde{\rho}) = \frac{\tanh(\beta\eta) + \tanh(\beta(\tilde{\rho} - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))}$$

$\bar{\rho} = H(\beta, \eta, \tilde{\rho})$ is now the physical density, entering the modified SIMP model:

$$E(\rho) = E_{\min} + (E_{\text{material}} - E_{\min}) \bar{\rho}^p$$

And in the volume constraint:

$$\int_{\Omega} \bar{\rho} d\Omega \leq V_{\max}$$

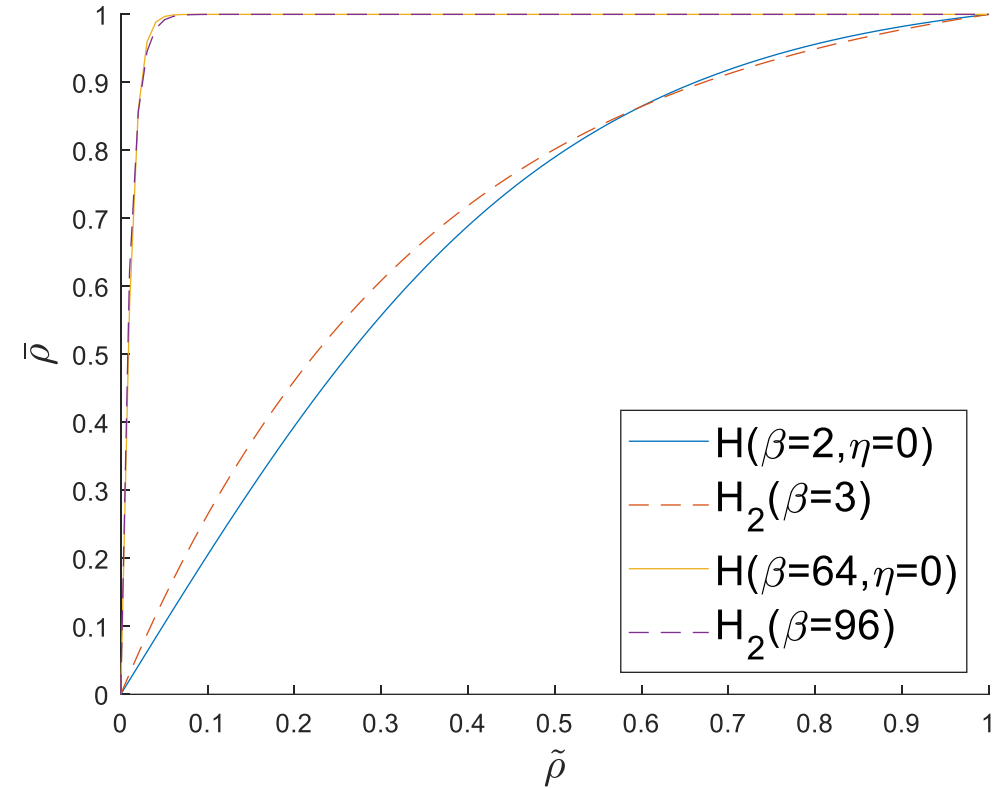
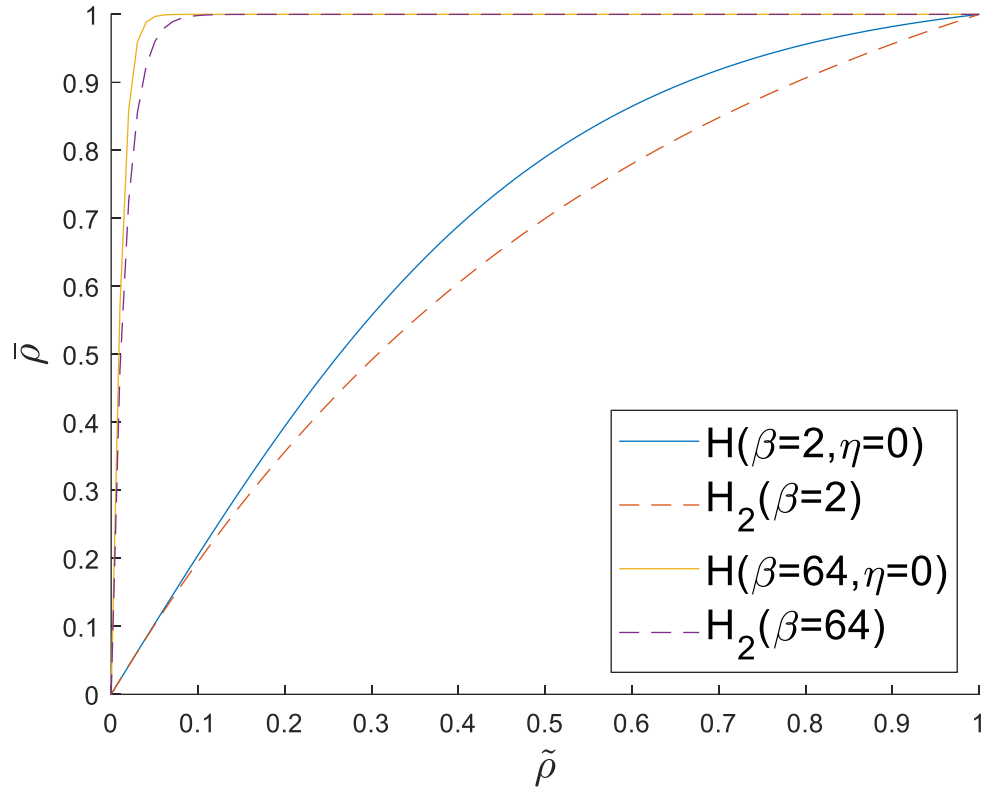
Projection

Another popular function for projection is to approximate the Heaviside step function with

$$H_2(\beta, \tilde{\rho}) = 1 - e^{-\beta \tilde{\rho}} + \tilde{\rho} e^{-\beta}$$

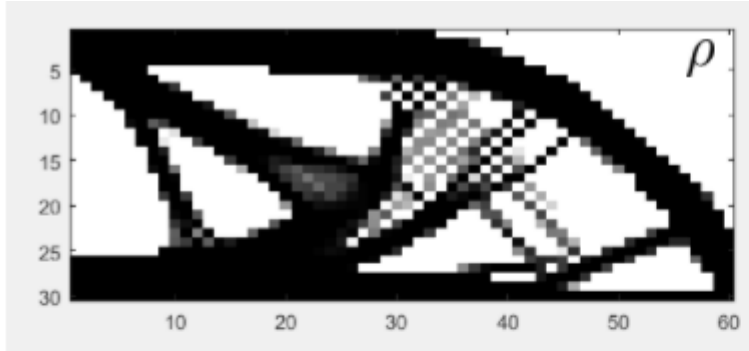
This function is similar to $H(\beta, \eta, \tilde{\rho})$ on the previous slide for $\eta = 0$.

If β is scaled in $H_2(\beta, \tilde{\rho})$, e.g. $H_2(1.5\beta, \tilde{\rho})$, the function is now very similar to $H(\beta, \eta = 0, \tilde{\rho})$.

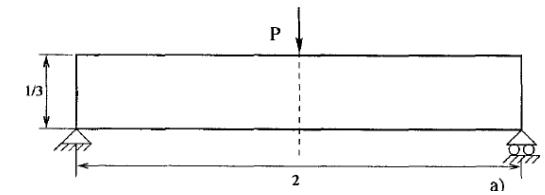
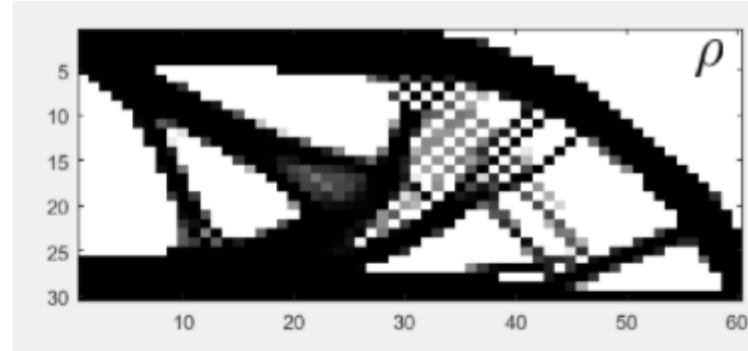


Filtering and Projection

Design Variables

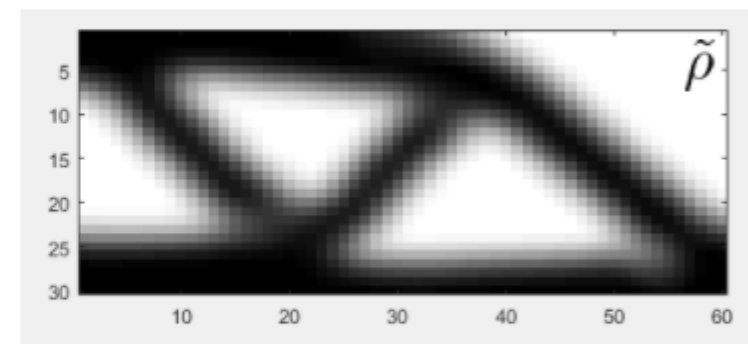
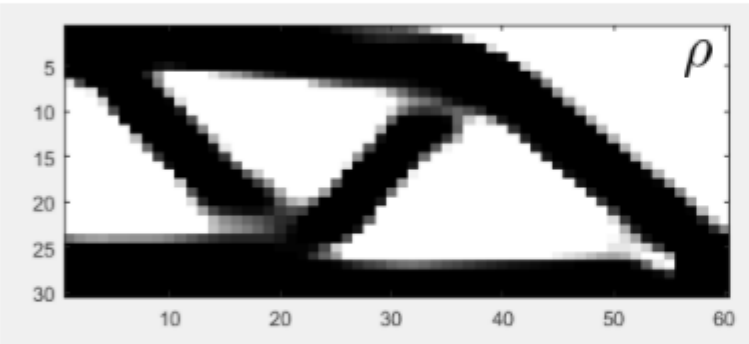


Variable used in SIMP equation and volume constraint.

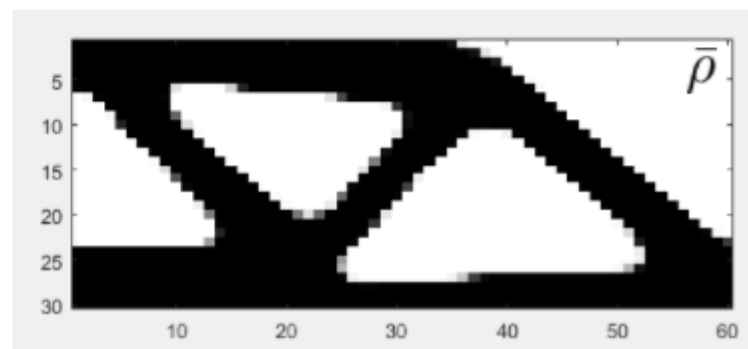
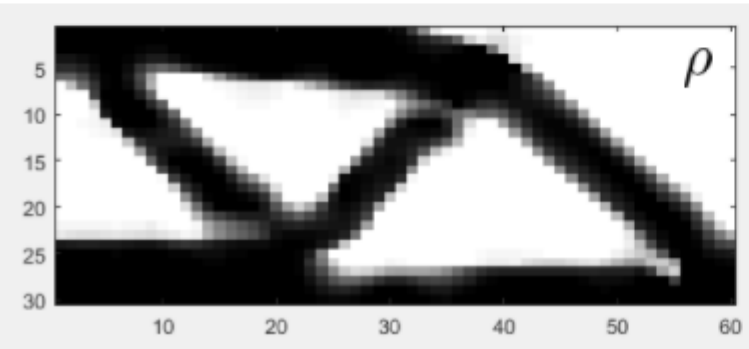


For full optimization:

Without filtering
nor projection.



With filtering only.



With filtering and
projection.

The maximum stiffness problem formulation

Choose an initial design field ρ .

Starting from the latest ρ , solve for $\beta = 1, \dots, \beta_{\max}$:

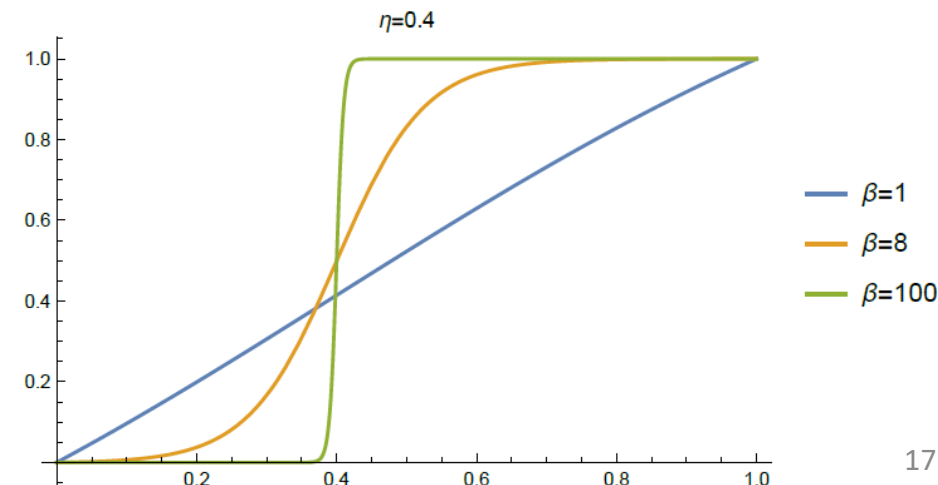
$$\begin{aligned} \min_{\rho_i} F(\rho(\mathbf{r})) &= \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} d\Omega \\ \text{such that } \sigma_{ij,j} &= f_i \text{ in } \Omega, \quad \sigma_{ij} n_j = T_i \text{ on } \partial\Omega^N, \quad u_i = u_i^D \text{ on } \partial\Omega^D \\ \epsilon_{ij} &= \frac{1}{2} (u_{i,j} + u_{j,i}) \\ \sigma_{ij} &= \lambda(E) \epsilon_{kk} \delta_{ij} + 2\mu(E) \epsilon_{ij} \\ -r^2 \nabla^2 \tilde{\rho} + \tilde{\rho} &= \rho, \quad \frac{\partial \tilde{\rho}}{\partial \mathbf{n}} = 0 \\ \bar{\rho} &= \frac{\tanh(\beta\eta) + \tanh(\beta(\tilde{\rho} - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))} \\ E(\mathbf{r}) &= E_{\min} + (E_{\text{material}} - E_{\min}) \bar{\rho}^p(\mathbf{r}) \\ \int_{\Omega} \bar{\rho}(\mathbf{r}) d\Omega &\leq V_{\max} \\ 0 &\leq \rho \leq 1 \end{aligned}$$

The problems for fixed β can be efficiently solved using MMA.

If β is large, the gradient component becomes ~ 0 when $\tilde{\rho}_i \gtrsim \eta$ or $\tilde{\rho}_i \lesssim \eta$.

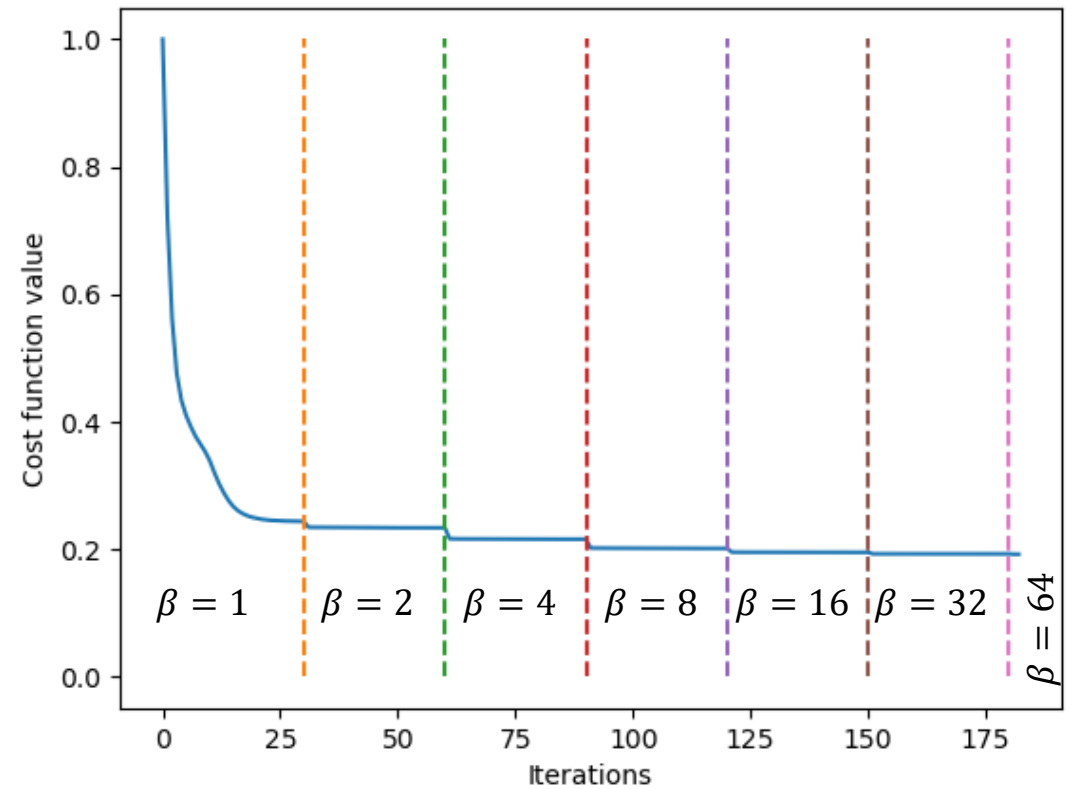
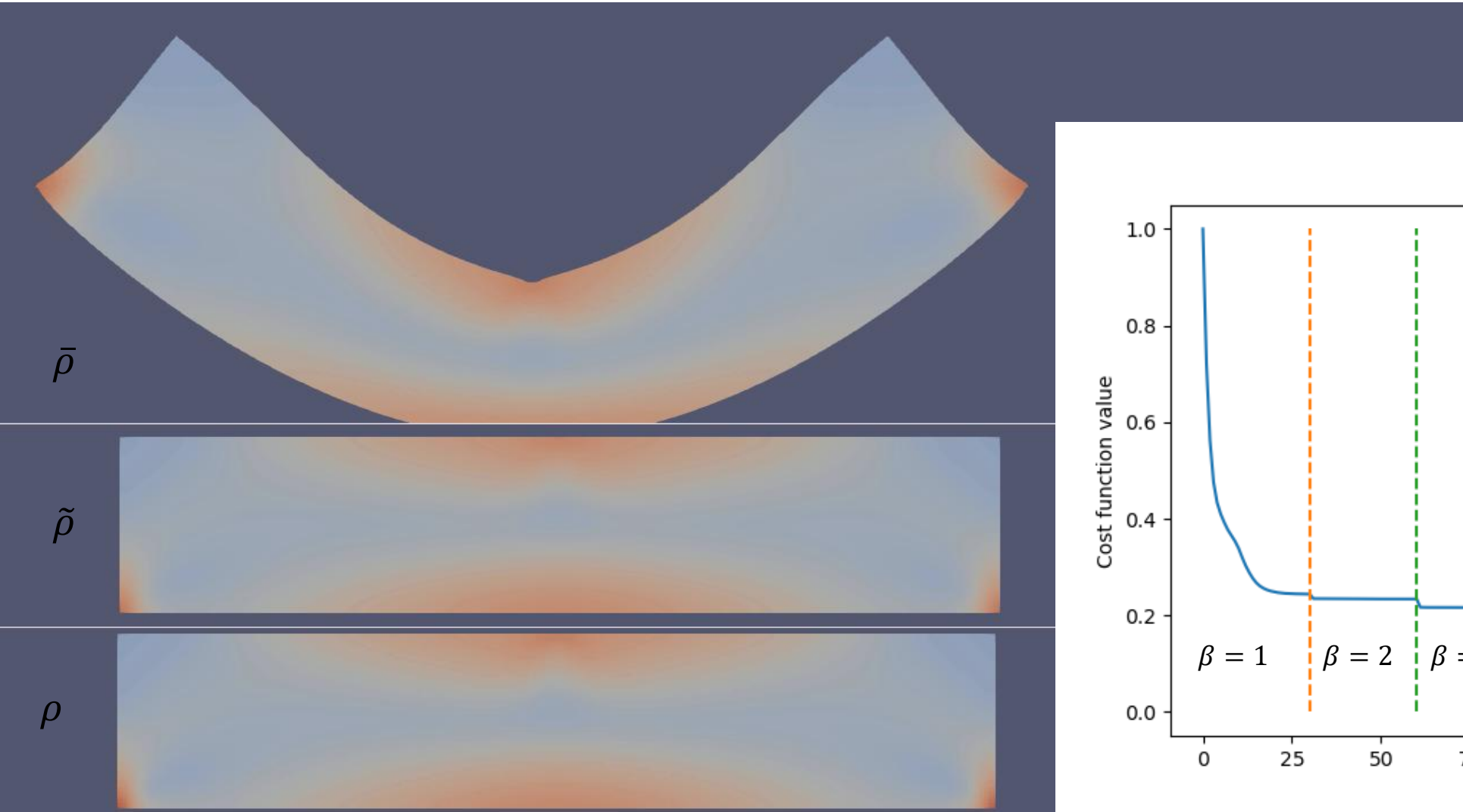
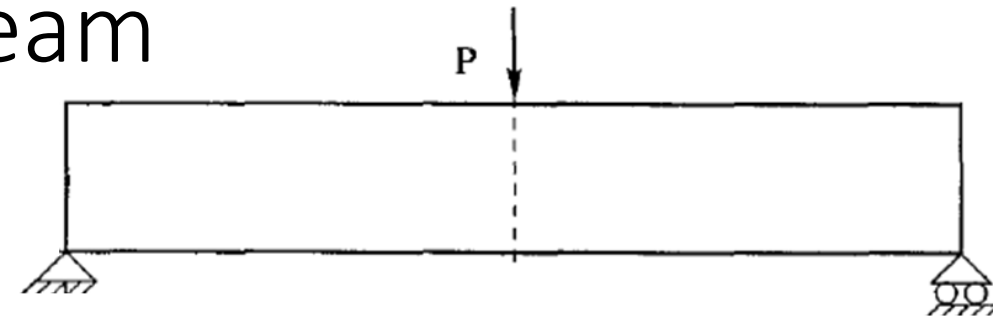
Raising β has the effect of "freezing" areas where $\bar{\rho}$ is either ~ 1 or ~ 0 .

The maximum value of β , $\beta_{\max}$, is usually chosen to be 32, 64, 128 or "thereabout".



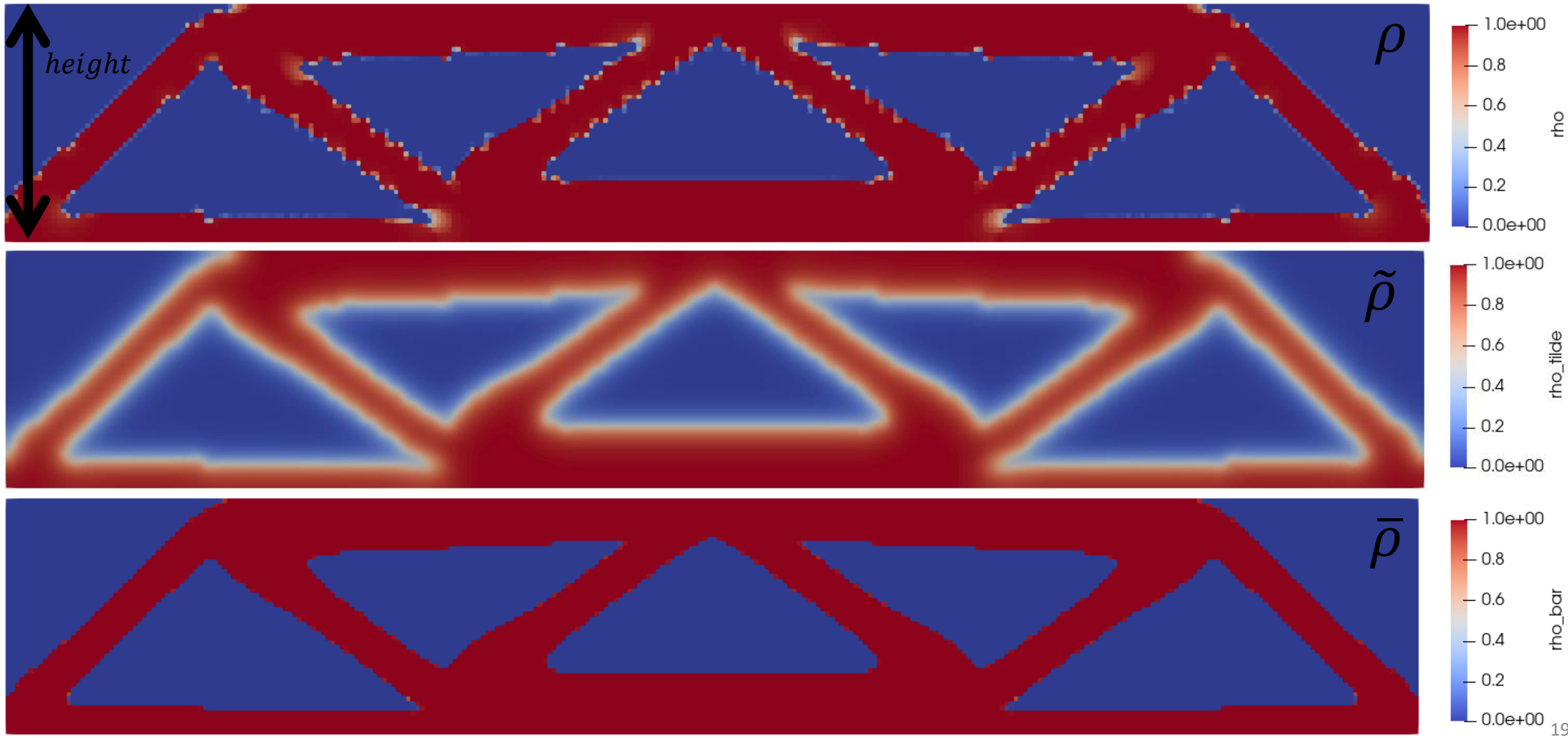
The classical example of the MBB beam

$$r = \frac{\text{height}/8}{2\sqrt{3}}, \quad \eta = 0.5, \quad V_{\max} = 0.5V_0$$



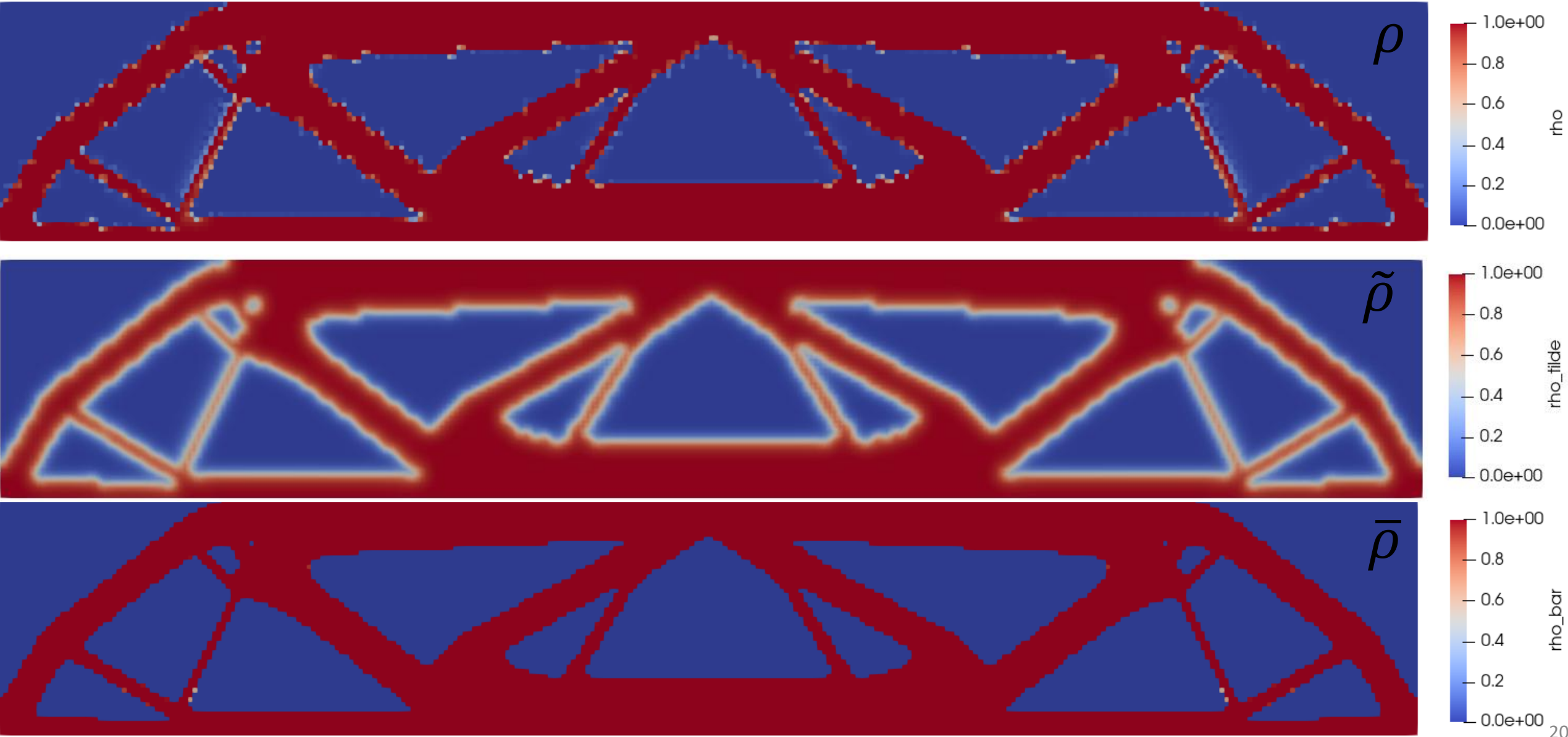
Final Average Strain Energy Density: 0.0025191017184563553 Final Volume fraction: 0.4999958594632129

$$r_{\min} = \frac{height}{8}, \quad r = r_{\min}/(2\sqrt{3}), \quad \eta = 0.5, \quad \beta = 64$$



Final Average Strain Energy Density: 0.0024842712317228906 Final Volume fraction: 0.4999935380123534

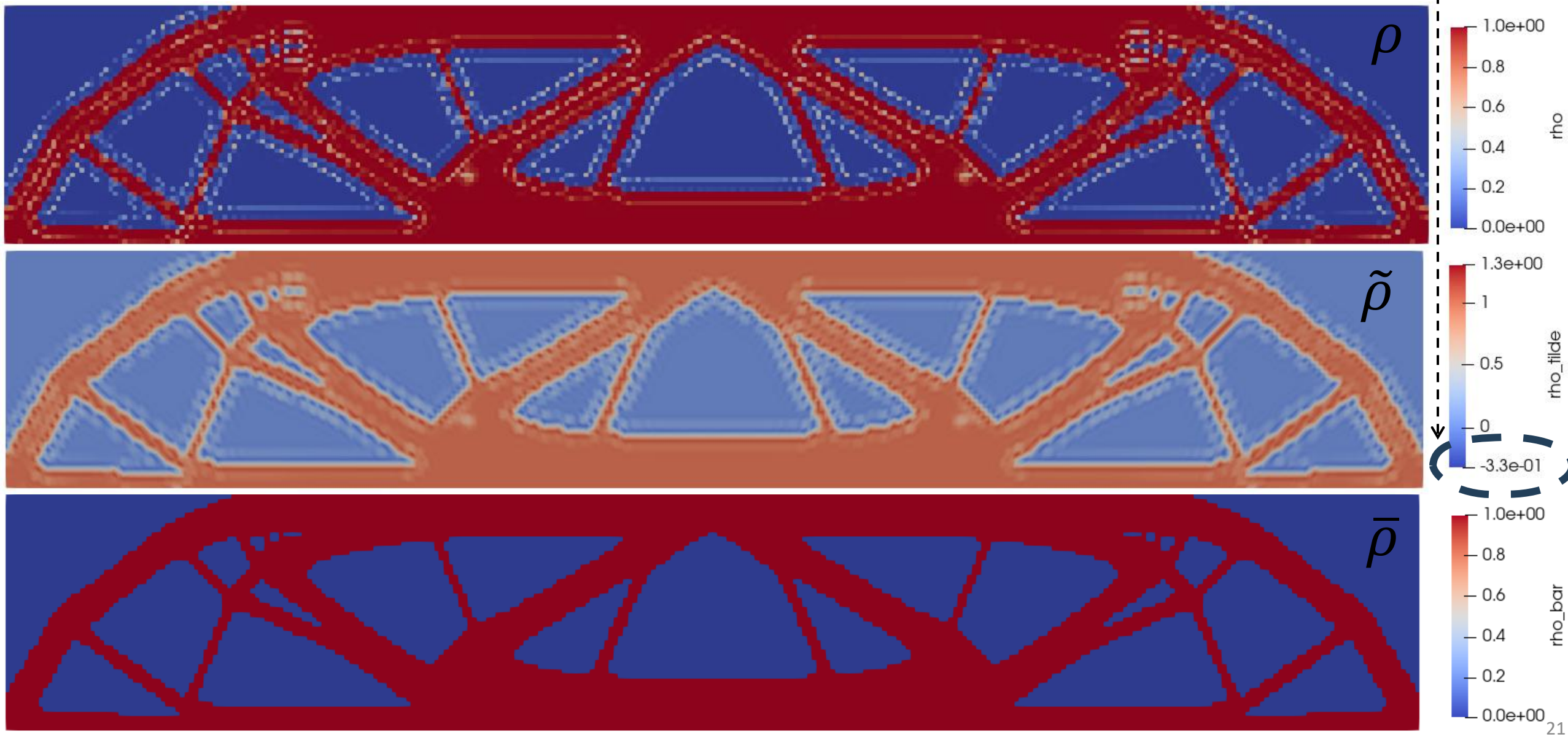
$$r_{\min} = \frac{height}{16}, \quad r = r_{\min}/(2\sqrt{3}), \quad \eta = 0.5, \quad \beta = 64$$



If $r \lesssim 2 \text{ meshsize}/(2\sqrt{3})$ in the Helmholtz filter (PDE filter), negative densities in the filtered field is often observed. This can lead to a negative $\bar{\rho}$, which can lead to a negative Youngs modulus and a solver failure.

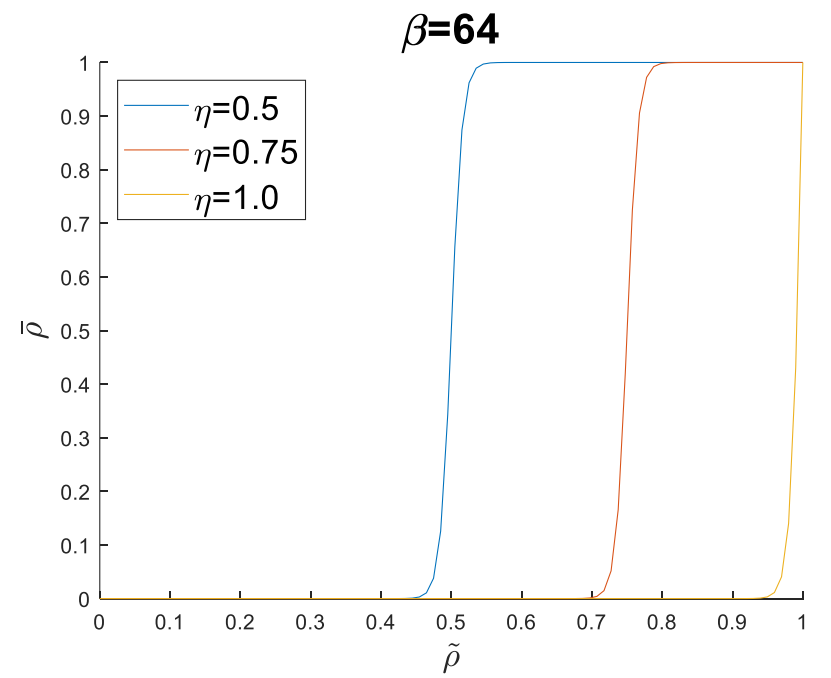
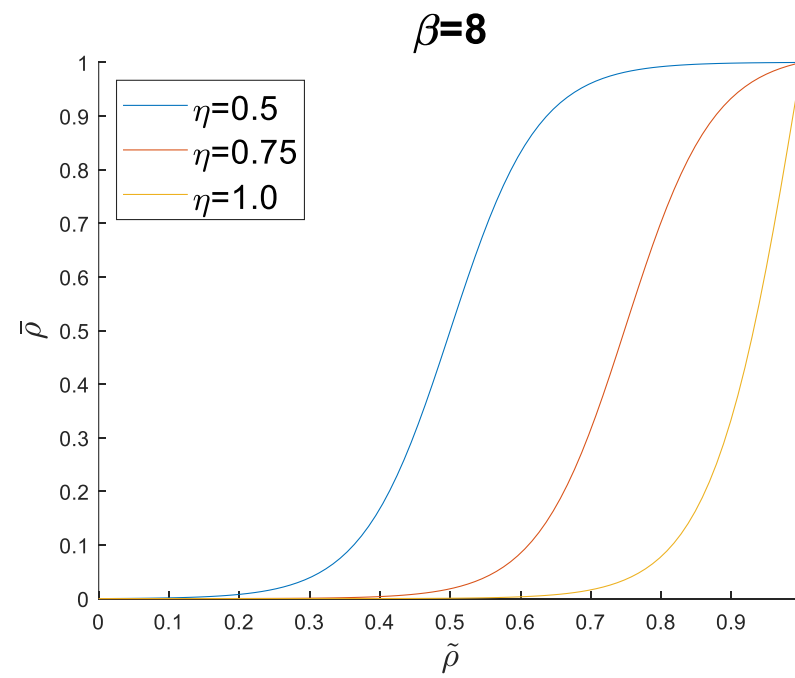
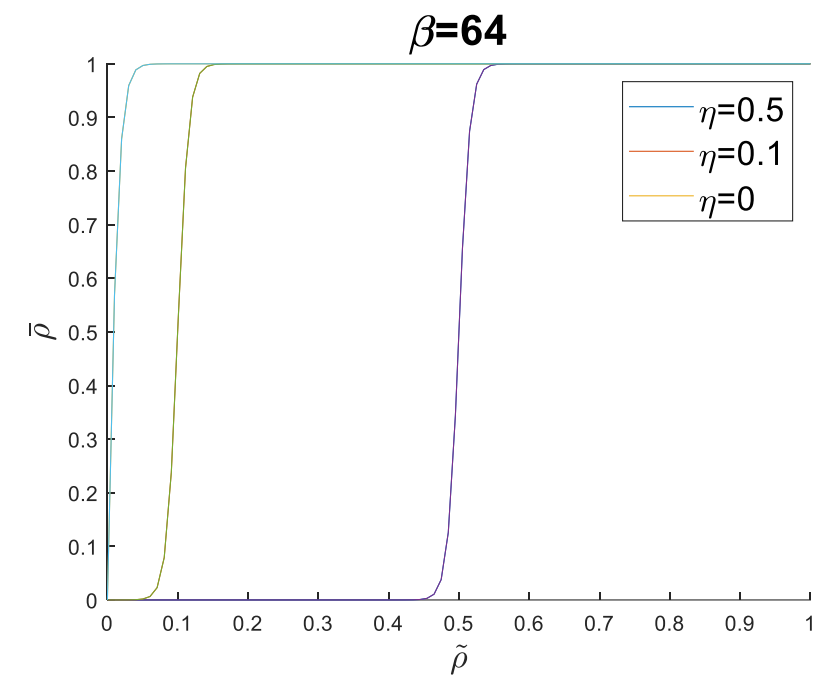
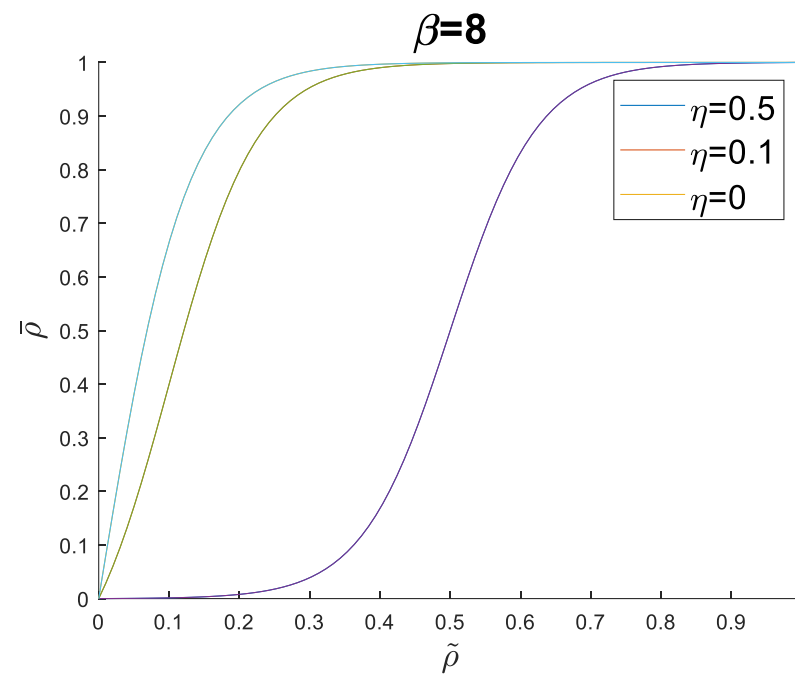
The example below, with $r_{\min} = \frac{\text{height}}{80}$, $r = r_{\min}/(2\sqrt{3})$, $\eta = 0.5$, $\beta = 64$, resulted in $\bar{\rho}$ staying positive, so it went “fine”.

Don't use too small filter radius ($r \lesssim 2 \text{ meshsize}/(2\sqrt{3})$) with the Helmholtz filter !!



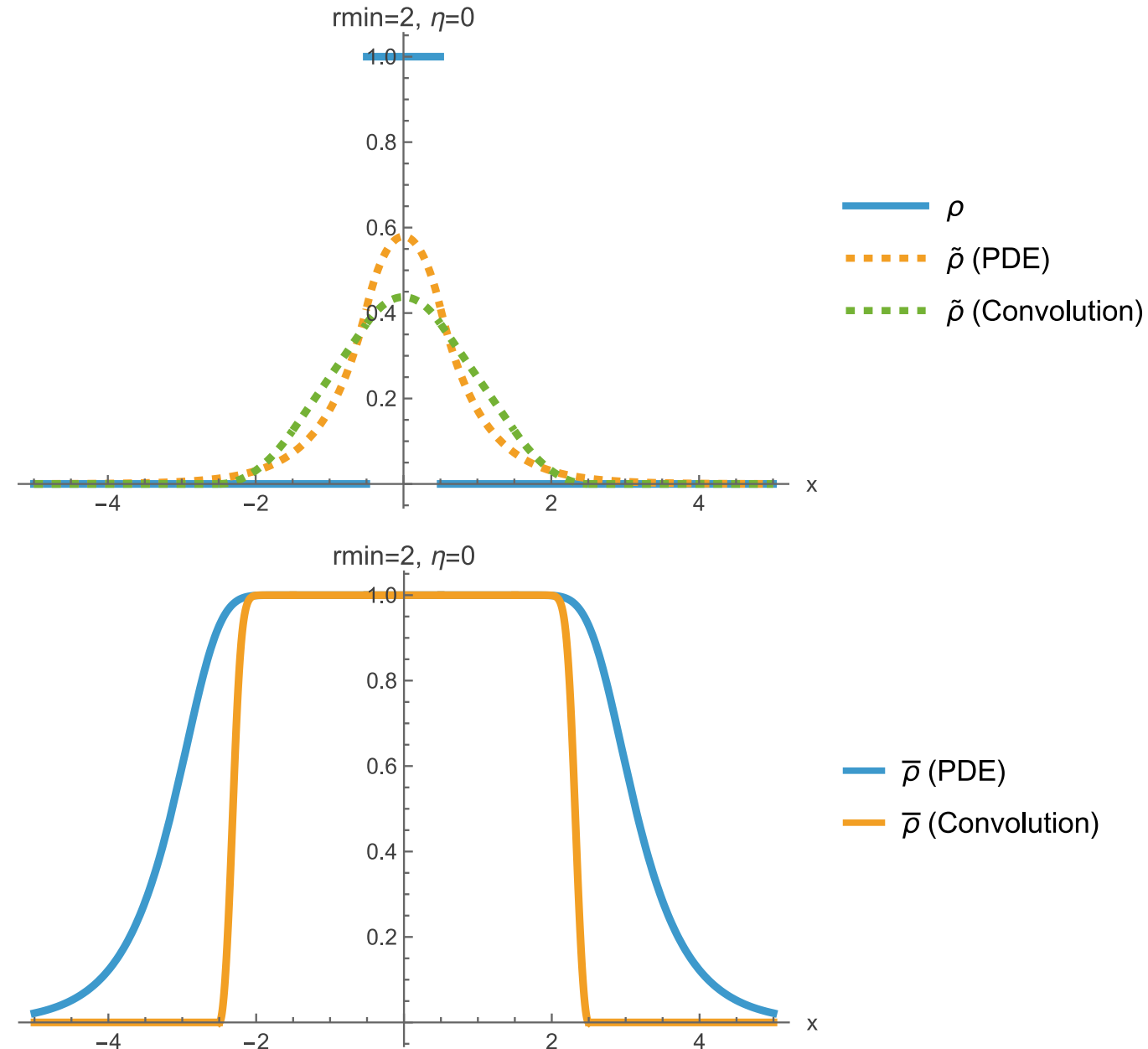
The role of η

- η controls the $\tilde{\rho}$ value, where the smoothed Heaviside function “jumps” from 0 to 1 at high β .
- When using a convolution filter, the values $\eta = 0$ and $\eta = 1$ can be used to get precise feature control.
- When using the PDE filter, $\eta = 0$ and $\eta = 1$ should be avoided.
- When using a PDE filter, $\eta = 0.5$ is very often chosen.



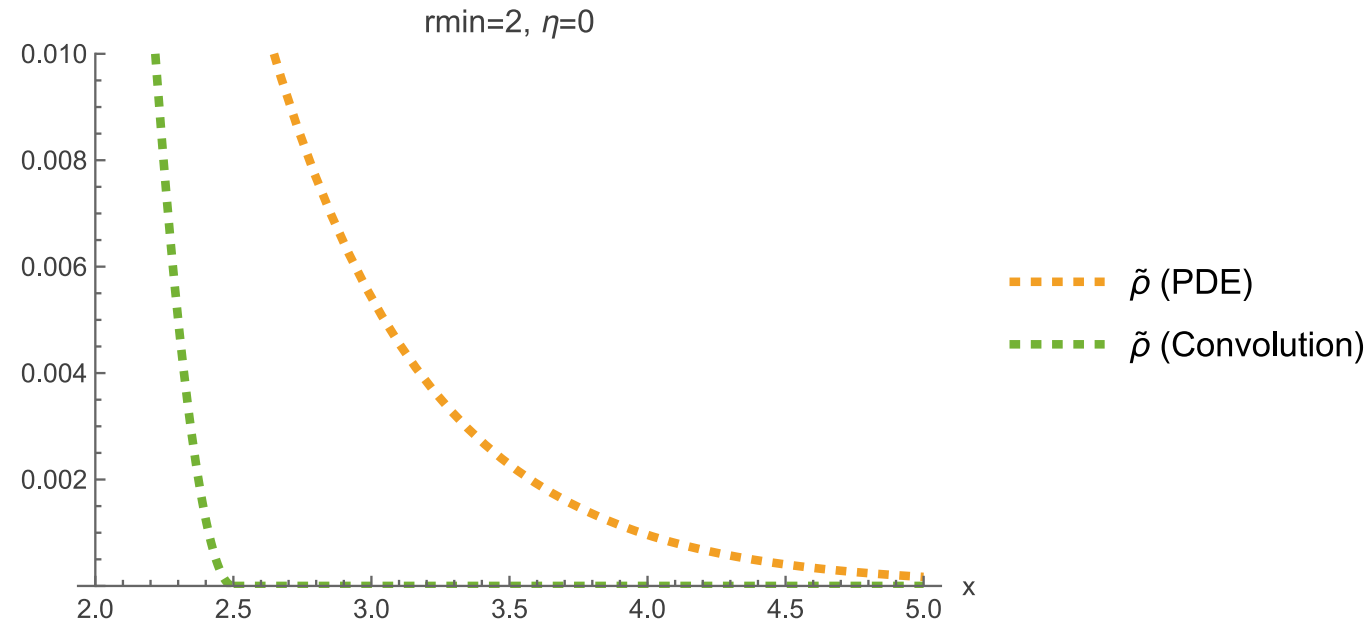
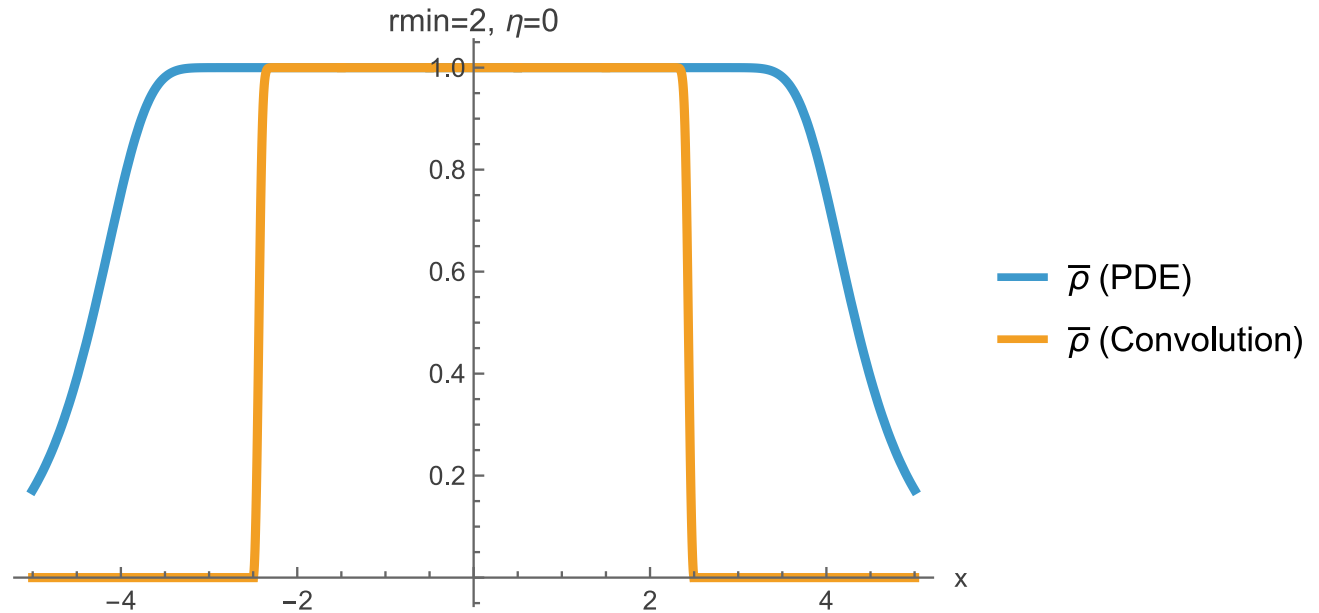
The $\eta = 0$ case: Control of solid regions.

- Consider an input ρ which is 1 inside a region of width 1 centered at $x=0$ and $\rho = 0$ outside this region.
- Top plot show the effect of the PDE filter and the convolution filter when $r_{\min} = 2$.
- Bottom plot show the filtered and projected densities for $\beta = 128$ and $\eta = 0$.
- The solid region with $\bar{\rho} \approx 1$ for the convolution filter is $2.5 \lesssim x \lesssim 2.5$.
- The solid region for the PDE filter is somewhat wider and “soft”.



The $\eta = 0$ case: Control of solid regions.

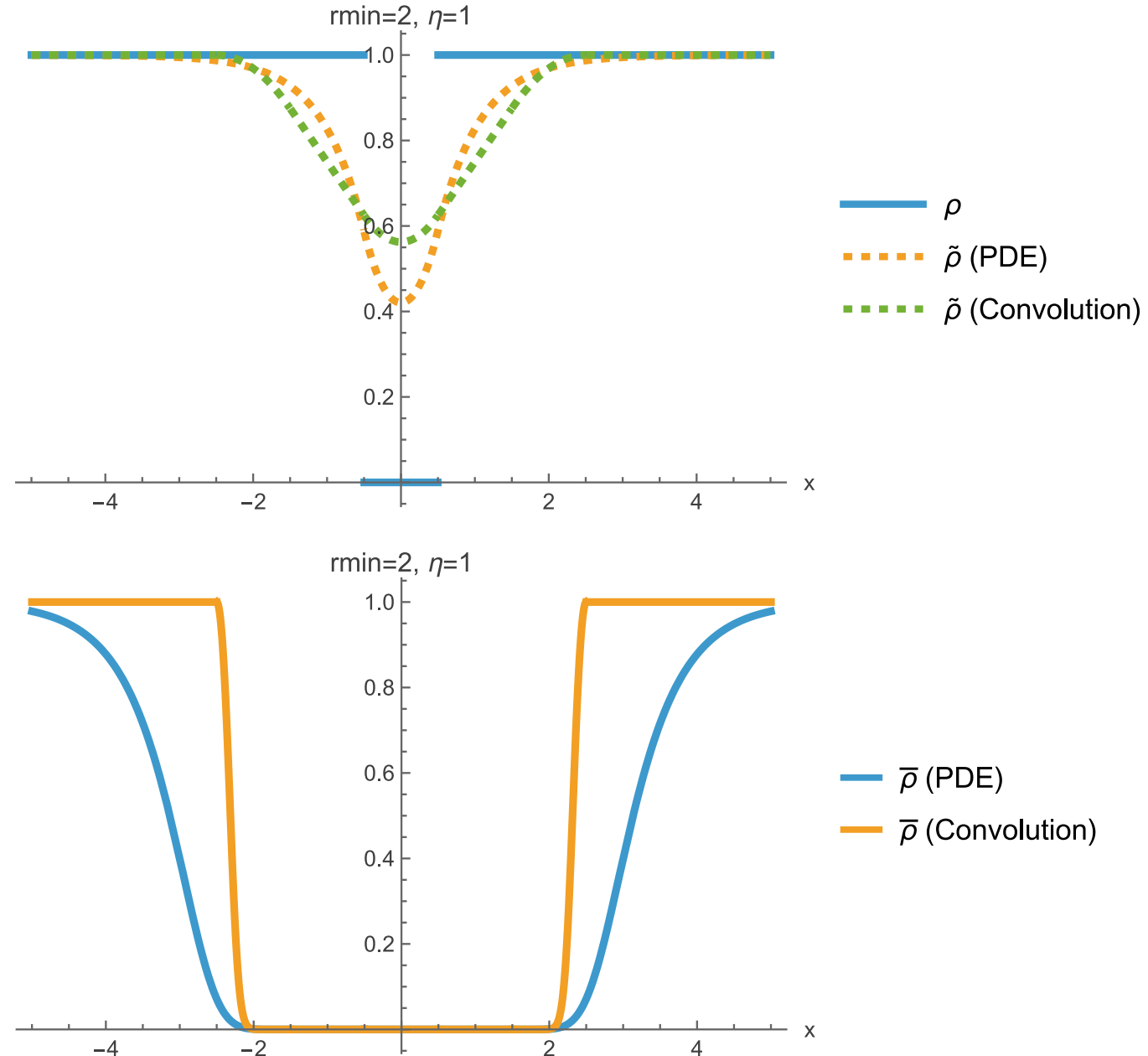
- At $\beta = 1024$, the material region is still $2.5 \lesssim x \lesssim 2.5$ for the convolution filter.
- For the PDE filter it has become much wider!
- Zooming in on the filtered densities, the convolution filter give $\tilde{\rho} = 0$ when $|x| > 2.5$.
- The PDE filter has non-zero values which gets projected to ~ 1 even at large x if β is high enough.
- Using $\eta = 0$ provide a length scale of $r_{\min}$ for solid regions for the convolution filter.
- **$\eta = 0$ should be avoided for the PDE filter.**



The $\eta = 1$ case:

Control of void region.

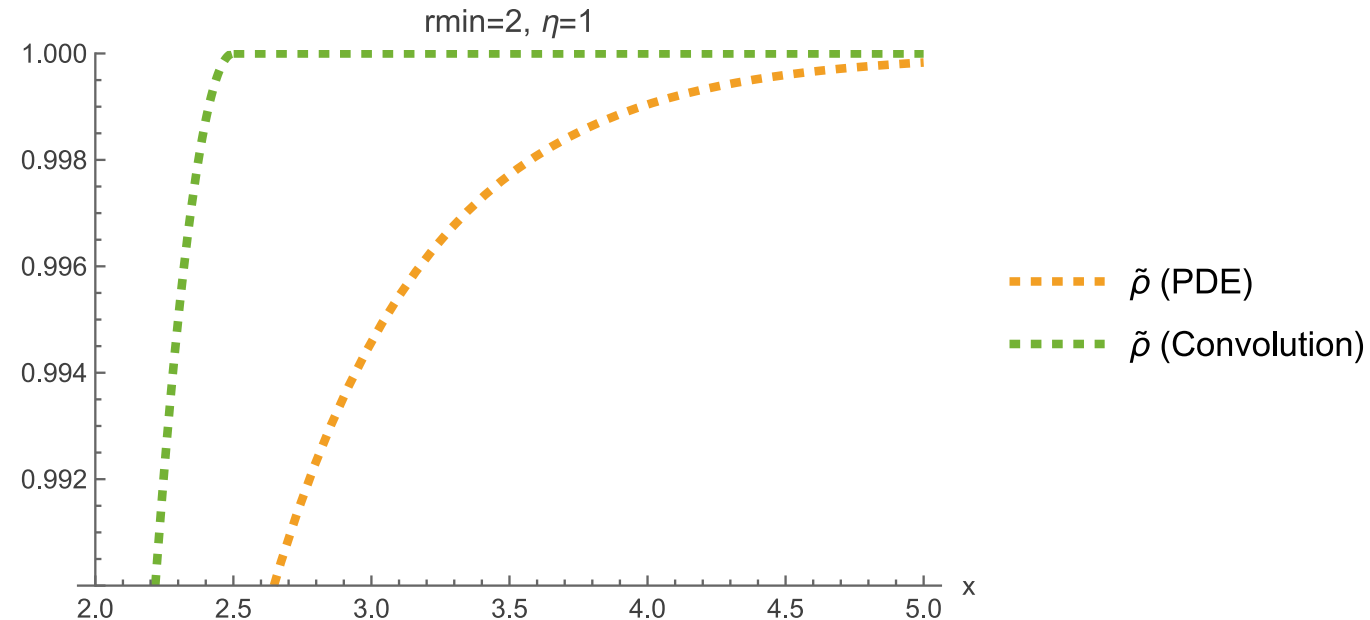
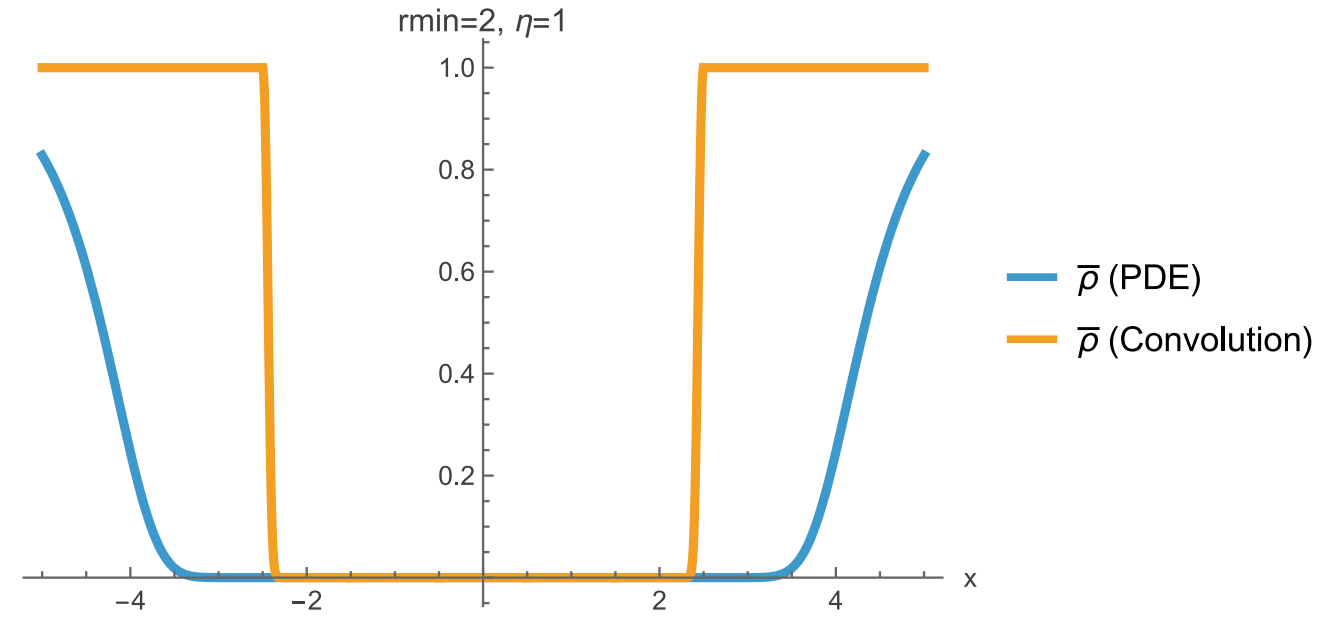
- Consider an input ρ which is 0 inside a region of width 1 centered at $x=0$ and $\rho = 1$ outside this region.
- Top plot show the effect of the PDE filter and the convolution filter when $r_{\min} = 2$.
- Bottom plot show the filtered and projected densities for $\beta = 128$ and $\eta = 1$.
- The void region with $\bar{\rho} \approx 0$ for the convolution filter is $2.5 \lesssim x \lesssim 2.5$.
- The void region for the PDE filter is somewhat wider and “soft”.



The $\eta = 1$ case:

Control of void region.

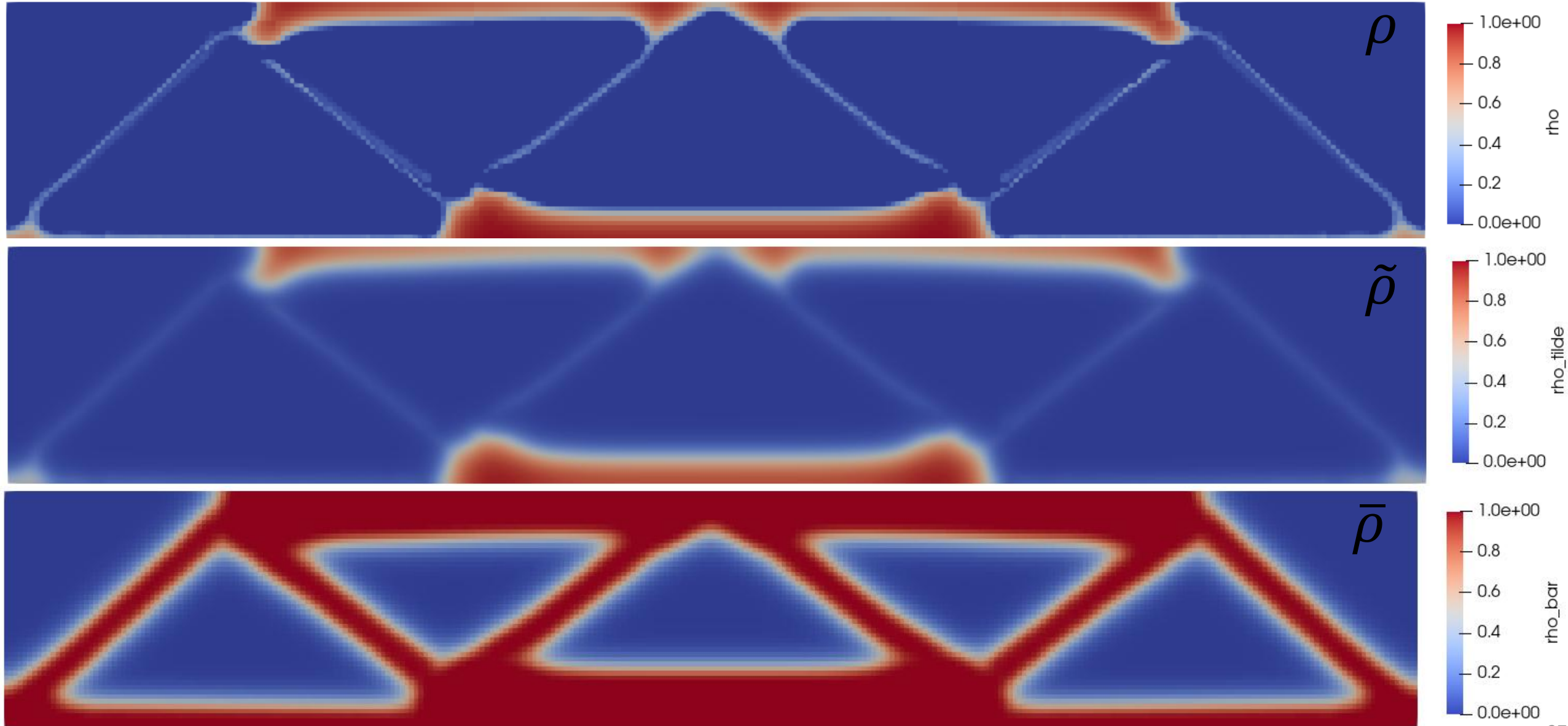
- At $\beta = 1024$, the void region is still $2.5 \lesssim x \lesssim 2.5$ for the convolution filter.
- For the PDE filter it has become much wider!
- Zooming in on the filtered densities, the convolution filter give $\tilde{\rho} = 1$ when $|x| > 2.5$.
- The PDE filter has non-zero values which gets projected to ~ 0 even at large x if β is high enough.
- Using $\eta = 1$ provide a length scale of $r_{\min}$ for void regions for the convolution filter.
- **$\eta = 1$ should be avoided for the PDE filter.**



Final Average Strain Energy Density: 0.0029376086767390324 Final Volume fraction: 0.4999791554254209

$$r_{\min} = \frac{\text{height}}{8}, \quad r = r_{\min}/(2\sqrt{3}), \quad \eta = 0, \quad \beta = 64$$

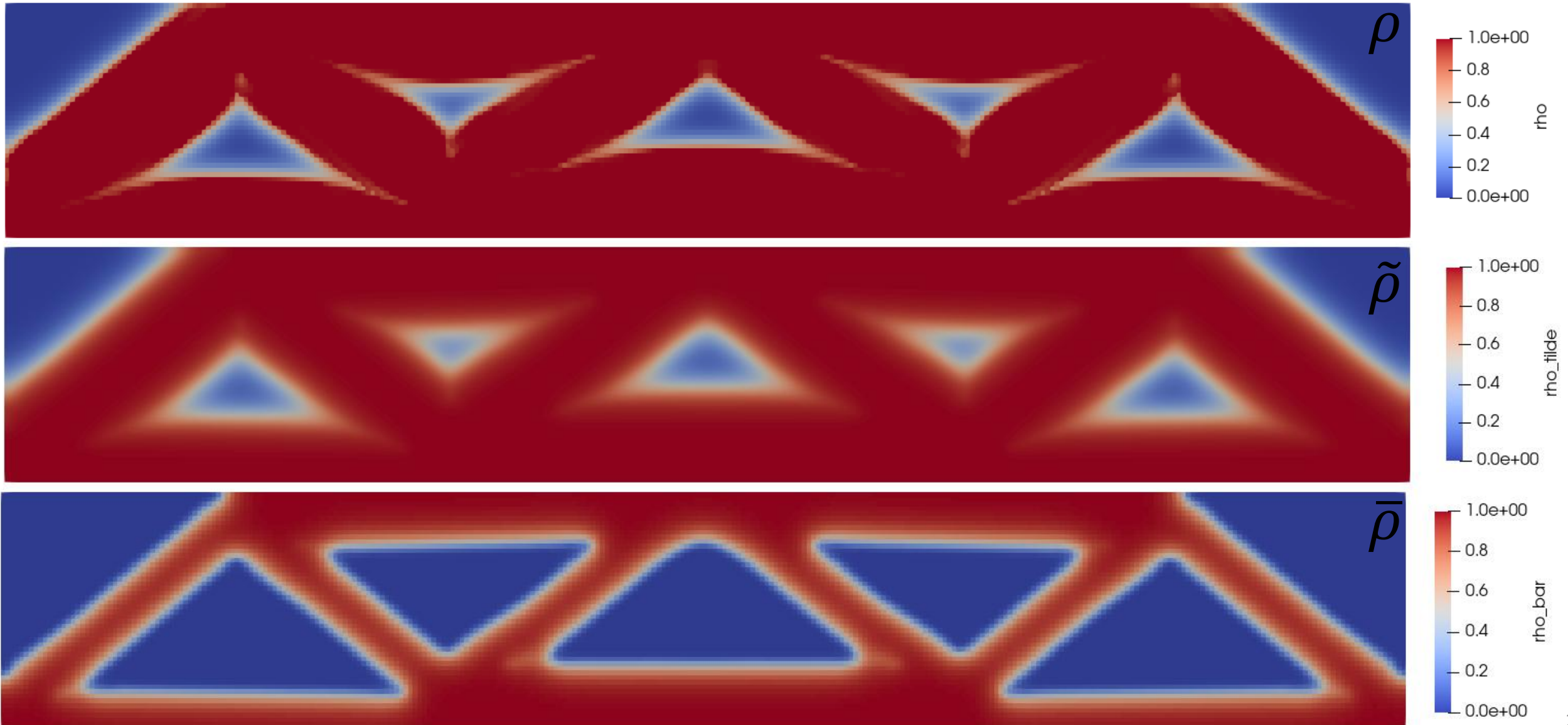
Soft regions is seen in the project density, even for high β .



Final Average Strain Energy Density: 0.0030109659303266026 Final Volume fraction: 0.4999929729897733

$$r_{\min} = \frac{\text{height}}{8}, \quad r = r_{\min}/(2\sqrt{3}), \quad \eta = 1, \quad \beta = 64$$

Soft regions is seen in the project density, even for high β .



A small program for Topology Optimization

```

from fenics import *
from fenics_adjoint import *
from mbb import MBB
from filter_and_project import FilterAndProject
import numpy as np
from mma import MMA_petsc

class topOpt():
    def __init__(self):
        # MPI rank
        self.rank=MPI.rank(MPI.comm_world)

        # Directory for output
        self.directory='topopt_elasticity_mbb/'

        # Some constants
        density = 1380      # Density, [kg/m^3]
        L = 1               # Length, [m]
        W = L/3             # Width, [m]
        g = 9.81            # Gravitational acceleration, [m/s^2]
        density = 1380      # Density, [kg/m^3]
        F = -density*L*W*W*g # Total force on boundary, [N]
        self.E0 = 3e9        # Youngs modulus, [Pa]
        self.nu = 0.38       # Poissons ration, [1]
        mesh_size=W/50      # Mesh size
        self.Volfrac=0.5
        self.VolMax=L*W

        R=W/8/(2*np.sqrt(3)) # Filter Radius ~ W/8
        #R=2*mesh_size/(2*np.sqrt(3)) # Minimum filter Radius
        self.Emin=self.E0/1e6 # Minimum E
        self.pen=3           # SIMP penalization

        self.mech=MBB(L,W,W,self.E0,self.nu,F,0*density,mesh_size)#,cell_type=CellType.Type.hexahedron)

        self.FP=FilterAndProject(R,self.mech.mesh)
        self.rho=interpolate(Constant(0.5),self.FP.Vd)
        self.rho.rename('rho','rho')
        self.rho_petsc=as_backend_type(self.rho.vector()).vec() # Pointer to rho's petsc data structure

        # Set up optimizer
        # Number of constraints
        self.ncon=1
        self.setUpOptimizer( self.rho_petsc )

```

A small program for Topology Optimization

```
def optimize(self):
    # Set loglevel to 30 to disable some fenics messages...
    set_log_level(30)
    # For output of optimization history
    self.iter=0
    # Save markings for inspection in paraview
    File(self.directory+'boundary_parts.pvd') << self.mech.boundary_parts
    # Output files
    self.u_fid=File(self.directory+'u.pvd')
    self.rho_fid=File(self.directory+'rho.pvd')
    self.rho_tilde_fid=File(self.directory+'rho_tilde.pvd')
    self.rho_bar_fid=File(self.directory+'rho_bar.pvd')
    self.nout=5

    self.beta=1
    self.eta=0.5
    betaMax=64
    while self.beta<=betaMax:
        if self.rank==0:
            print('beta=',self.beta)
            self.setUpFunctionals(self.beta,self.eta)
            self.mma.solve( self.rho_petsc )
            self.beta=self.beta*2
    self.beta=self.beta/2
    self.plot_k(self.mma.x,True)
    (u,rho_bar,rho_tilde)=self.Forward(self.beta,self.eta)
    eps=self.mech.epsilon(u)
    sig=self.mech.sigma(u,eps)
    Energy=float(assemble(0.5*inner(sig,eps)*dx,annotate=False))
    Volume=float(assemble((Constant(1)+1e-9*rho_bar)*dx))
    UsedVolume=float(assemble(rho_bar*dx))
    return (Energy,UsedVolume,Volume)

if __name__ == "__main__":
    t=topOpt()
    if t.rank==0:
        print('DOFs in mechanics problem: ',t.mech.V.dim())
    (Energy,UsedVolume,Volume)=t.optimize()
    if t.rank==0:
        print('Final Average Strain Energy Density: ',Energy/Volume,'Final Volume fraction:',UsedVolume/Volume)
```

```

def setUpOptimizer(self,x0):
    """Creates and initializes MMA"""
    self.mma=MMA_petsc(x0,self.ncon,f=self.f,g=self.g,plot_k=self.plot_k)
    self.mma.xmin[:]=0.0
    self.mma.xmax[:]=1.0

    # MMA setup
    self.mma.move=0.5
    self.mma.xtol=0.01
    self.mma.ftol=0.001
    self.mma.lmax=5
    self.mma.kmax=30
    self.mma.kmin=1
    self.mma.it=0
    self.mma.mma_timing=False

def Forward(self,beta,eta):
    rho_tilde=self.FP.filter(self.rho)
    rho_bar,_=self.FP.Heavi_project(beta,eta,rho_tilde)
    #rho_tilde=self.rho # Use to "turn off" filtering
    #rho_bar=rho_tilde # and projection
    E=self.Emin+pow(rho_bar,self.pen)*(self.E0-self.Emin) # SIMP E
    self.mech.E=E
    self.mech.mask=rho_bar
    u=self.mech.solve()
    return (u,rho_bar,rho_tilde)

def plot_k(self,x,force_plot=False):
    """Used for output during optimization"""
    if force_plot or self.iter % self.nout == 0:
        if not force_plot:
            self.rho.vector().set_local(x.getArray()[:])
            self.rho.vector().apply('')

        (u,rho_bar,rho_tilde)=self.Forward(self.beta,self.eta)

        self.u_fid << u,self.iter
        self.rho_fid << self.rho,self.iter
        self.rho_tilde_fid << rho_tilde,self.iter
        self.rho_bar_fid << rho_bar,self.iter
    self.iter+=1

```

A small program for Topology Optimization

A small program for Topology Optimization

Middle part of the code

```
def setUpFunctionals(self,beta,eta):
    from pyadjoint.tape import annotate_tape
    while not annotate_tape():
        continue_annotation() # Run continue until we actually start annotating...
    tape=get_working_tape()
    tape.clear_tape()
    # Run forward model to record on tape
    (u,rho_bar,rho_tilde)=self.Forward(beta,eta)
    eps=self.mech.epsilon(u)
    sig=self.mech.sigma(u,eps)
    # Using annotate=False to not record this step, such that J0 does not depend on rho
    if beta==1:
        self.J0=float(assemble(0.5*inner(sig,eps)*dx,annotate=False))
    # Make functionals
    m=Control(self.rho)
    self.J=0.5*inner(sig,eps)/self.J0*dx
    self.V=(rho_bar-self.Volfrac)/self.VolMax*dx
    J=assemble(self.J)
    V=assemble(self.V)
    #self.Jhat = ReducedFunctional(J,m,tape=tape)
    #self.Vhat = ReducedFunctional(V,m,tape=tape)
    # Copy tape, to be able to run optimize such that e.g. mechanics is not solved for when calculating volume.
    tapeJ=tape.copy()
    tapeV=tape.copy()
    self.Jhat = ReducedFunctional(J,m,tape=tapeJ)
    self.Vhat = ReducedFunctional(V,m,tape=tapeV)
    tapeJ.optimize(controls=[m],functionals=[J])
    tapeV.optimize(controls=[m],functionals=[V])
    while annotate_tape():
        pause_annotation() # Run pause until we actually pause annotating...
```

```
def f(self,x):
    """Define function for MMA that calculates the vector function f"""
    self.rho.vector().set_local(x.getArray()[:])
    self.rho.vector().apply('')
```

```
J=self.Jhat(self.rho)
V=self.Vhat(self.rho)
return np.array([J,V])
```

```
def g(self,x):
    """Define function for MMA that calculates the gradient (Jacobian) of f"""
    self.rho.vector().set_local(x.getArray()[:])
    self.rho.vector().apply('')
    dc=self.Jhat.derivative().vector().get_local()
    dv=self.Vhat.derivative().vector().get_local()
    return np.array([dc,dv])
```

The topic of next week!


```
from fenics import *
from fenics_adjoint import *
```

Part of the FilterAndProject class

```
class FilterAndProject:
    def __init__(self, R, mesh):
        self.Vc=FunctionSpace(mesh, 'Lagrange', 1)
        self.Vd=FunctionSpace(mesh, 'DG', 0)
        self.R=float(R)

    def filter(self, rho):
        trial=TrialFunction(self.Vc)
        test=TestFunction(self.Vc)
        F=(inner(pow(self.R,2)*grad(trial),grad(test))+trial*test)*dx - rho*test*dx
        a=lhs(F)
        L=rhs(F)
        rho_tilde=Function(self.Vc,name='rho_tilde')
        solve(a == L, rho_tilde, solver_parameters={"linear_solver": "cg", "preconditioner": "hype_amg"})
        return rho_tilde

    def tanh(self, x):
        # tanh(x) = (exp(x)-exp(-x))/(exp(x)+exp(-x)) = (1-exp(-2*x))/(1+exp(-2*x)) = 2/(1+exp(-2*x))-1
        return 2/(1+exp(-2*x))-1

    def Heavi(self, beta, eta, x, th=None):
        if th is None:
            th=self.tanh
        return (th(beta*eta)+th(beta*(x-eta)))/(th(beta*eta)+th(beta*(1-eta)))

    def project_Vd(self, u):
        # Calculates L2 projection of u onto function space Vd.
        trial=TrialFunction(self.Vd)
        test=TestFunction(self.Vd)
        a = inner(test, trial)*dx
        L = inner(test, u)*dx
        pu=Function(self.Vd,name='rho_bar')
        solve(a == L, pu, solver_parameters={"linear_solver": "cg", "preconditioner": "hype_amg"})
        return pu

    def Heavi_project(self, beta, eta, rho_tilde):
        # Projects rho_tilde to Vd and then projects H(u) to Vd
        # Without this 2-stage projection, 1-element gradients from the functions in Vc
        # will create smoothed steps in rho_bar.
        u=self.project_Vd(rho_tilde)
        rho_bar=self.project_Vd(self.Heavi(beta, eta, u))
        return rho_bar, u
```

A small program for Topology Optimization

A small program for Topology Optimization

Part of the MBB class

```

class MBB(CantileverBeam):
    def __init__(self,L,Wy,Wz,E,nu,F,density,mesh_size,cell_type=CellType.Type.quadrilateral):
        CantileverBeam.__init__(self,L,Wy,Wz,E,nu,F,density,mesh_size,cell_type)
        # Define new boundary condition
        self.boundary_parts.set_all(0)

class XSymmetry_boundary(SubDomain):
    def inside(self,x,on_boundary):
        return on_boundary and near(x[0],0)

class Roll_boundary(SubDomain):
    def __init__(self,Lx,Ly,rollersize):
        self.Lx=Lx
        self.Ly=Ly
        self.RollerSize=rollersize
        SubDomain.__init__(self)
    def inside(self,x,on_boundary):
        tol = 1E-10
        return on_boundary and near(x[1],0) and (x[0]>=(self.Lx-self.RollerSize-tol) and x[0]<=(self.Lx+tol))

class Load_boundary(SubDomain):
    def __init__(self,Lx,Ly,loadsize):
        self.Lx=Lx
        self.Ly=Ly
        self.LoadSize=loadsize
        SubDomain.__init__(self)
    def inside(self,x,on_boundary):
        tol = 1E-10
        return on_boundary and near(x[1],self.Ly) and (x[0]>=(0.0-tol) and x[0]<=(self.LoadSize+tol))

loadsize=self.mesh_size*2
load_boundary=Load_boundary(L,Wy,loadsize)
load_boundary.mark(self.boundary_parts,1)
xsym_boundary=XSymmetry_boundary()
xsym_boundary.mark(self.boundary_parts,2)
roll_boundary=Roll_boundary(L,Wy,self.mesh_size)
roll_boundary.mark(self.boundary_parts,3)

# Redefine traction size
self.traction = self.traction*(Wy*Wz)/loadsize

self.ds=Measure('ds',domain=self.mesh,subdomain_data=self.boundary_parts)
if self.dim==2:
    self.bc=[DirichletBC(self.V.sub(0), 0, self.boundary_parts,2),DirichletBC(self.V.sub(1), 0, self.boundary_parts,3)]
else:
    self.bc=[DirichletBC(self.V.sub(0), 0, self.boundary_parts,2),DirichletBC(self.V.sub(1), 0, self.boundary_parts,3),
              DirichletBC(self.V.sub(2), 0, self.boundary_parts,3)]
    
```

